

数字系统设计

Digital System Design

-----微处理器设计3

王维东

Weidong Wang

浙江大学信息与电子工程学院

College of Information Science & Electronic Engineering

信息与通信网络工程研究所

Zhejiang University

Spring 2026 – Lec. #14

任课教师

- 王维东
- 浙江大学信息与电子工程学院, 信电楼306
- 邮箱: wdwang@zju.edu.cn
- College of Information Science & Electronic Engineering
- Zhejiang University, Hangzhou, 310027
- Tel: 86-571-87953170 (O)
- Mobile: 13605812196

TA:

钟睿, 13687791509; 22331166@zju.edu.cn;

Office Hours: 玉泉信电楼 308室（可以钉钉、微信或邮件联系）.

Prerequisites预修课程

- 电子电路基础
- 电子线路
- C语言
- How to learn this Course?
 - Not only listening, thinking and waiting
 - But Exercise, Simulation, Practice!

课程简介

- 课程代码: **671C0050**
- 参考书
 - 阎石,《数字电子技术基础》,第6版,高等教育出版社,2016.
 - 王金明著,《数字系统设计与Verilog HDL》,电子工业出版社,第8版
 - 《数字系统设计-补充讲义》,浙江大学自编讲义
 - Stanford 大学 108A课程notes.
 - R.H.Katz, G.Borriello, *Contemporary Logic Design*, second edition,电子工业出版社,2005.
 - M.M.Mano, 数字设计(第四版),电子工业出版社,2010.
 - » <http://www.prenhall.com/mano>

Other Course Info

- **Website:**

- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- **学在浙大: 数字系统设计**
- Check frequently

- **答疑**

- 玉泉信电楼308室/周一周四下午2:30-5:30
- 上课课间、课后均可
- Email, 钉钉/微信群/数字系统设计, 短信均可

Grading (考核)

Final grades will be computed approximately as follows:

- 平时（含课程作业、期中考试、小测验、Project、出勤等） **40%**

- 期中考试 **10%**
- Homework Sets **15% (12~20)%**
- Project **5% (5~7)%**
- Class Room Check+小测验等 **10% (7~10)%**
- 作业每周一上交, **截止期为课后一周内**有效。

- **Project**

- Project-1必做 (1 members)
- Project-2可选(成绩加分1%~5%, 但不超过平时成绩范围)

- Finial Exam期末**闭卷**考试 - **60%**

授课时间和地点:

- 2025年春夏学期,
 - 周一上午, 第1~2节(8:00-9:35)
 - 星期四上午, 第6~7节(13:25-13:00)
- 地点: 紫金港西2-415(多)
- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- 学在浙大/数字系统设计
- 钉钉/微信/数字系统设计-答疑

课程结构

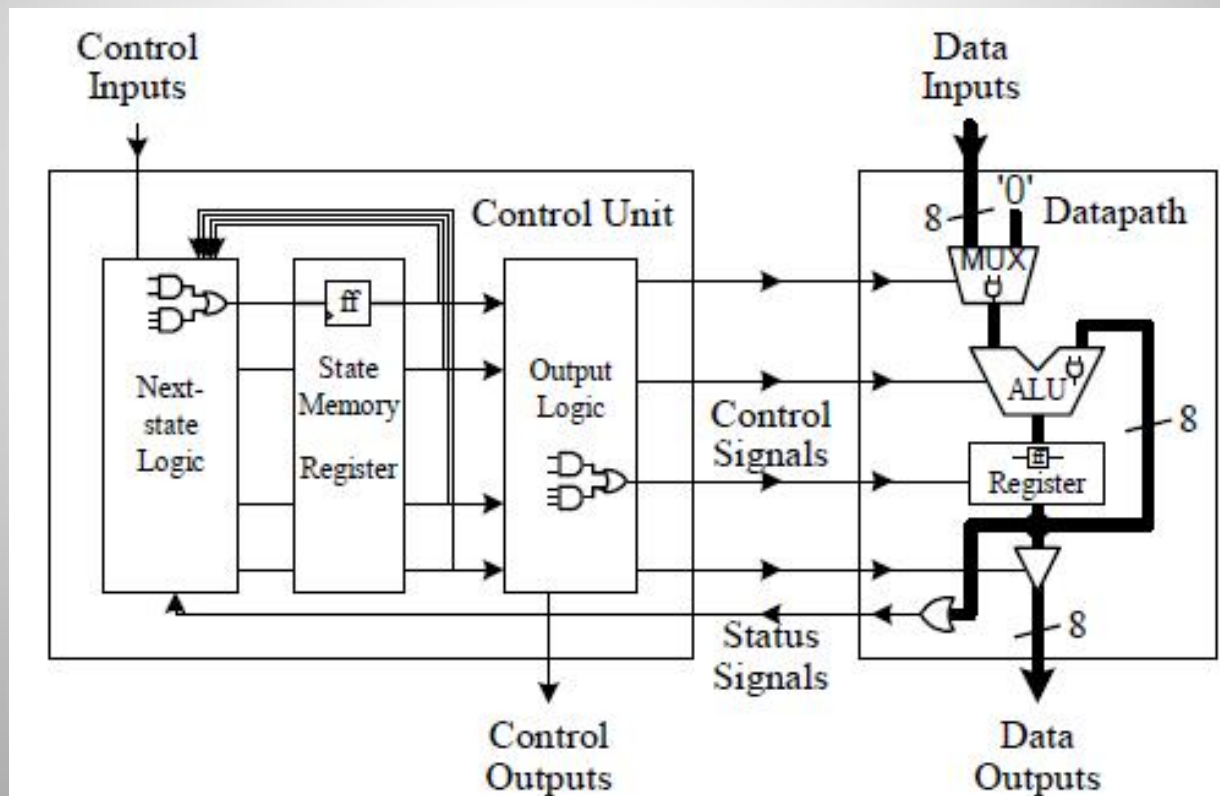
- 数字理论知识(必备)
 - 数字系统和编码、逻辑代数、门电路
- 数字电路分析与设计
 - 组合逻辑电路
 - 触发器、半导体存贮器、可编程器件
 - 时序逻辑电路
- 脉冲电路与接口
- 控制器与数字系统
 - 状态机
 - 控制器
 - 微码控制器
 - 测试和验证
- 微处理器简介
 - 指令集
 - **4位CPU**介绍

讲义第十章 微处理器设计3

-----微码型微处理器

微处理器内部结构

- 微处理器的两个主要部分：
 - 控制单元
 - » 用于控制数据通路的所有操作，实现微处理器运算的正确性
 - 数据通路
 - » 主要包括运算单元ALU、存储单元(寄存器)及其相互连接



微处理器内部结构

微处理器的三个执行步骤

- 指令周期

- (1)取指

- » 即从程序存储器中把PC所指向的指令取出, 拷贝到IR;
 - » PC+1指向下一条指令的地址;

- (2)译指

- » 从IR中提取出操作码, 对指令所要做的操作进行翻译;
 - » 决定当前执行那一条指令, 跳转到那一个对应的状态去;

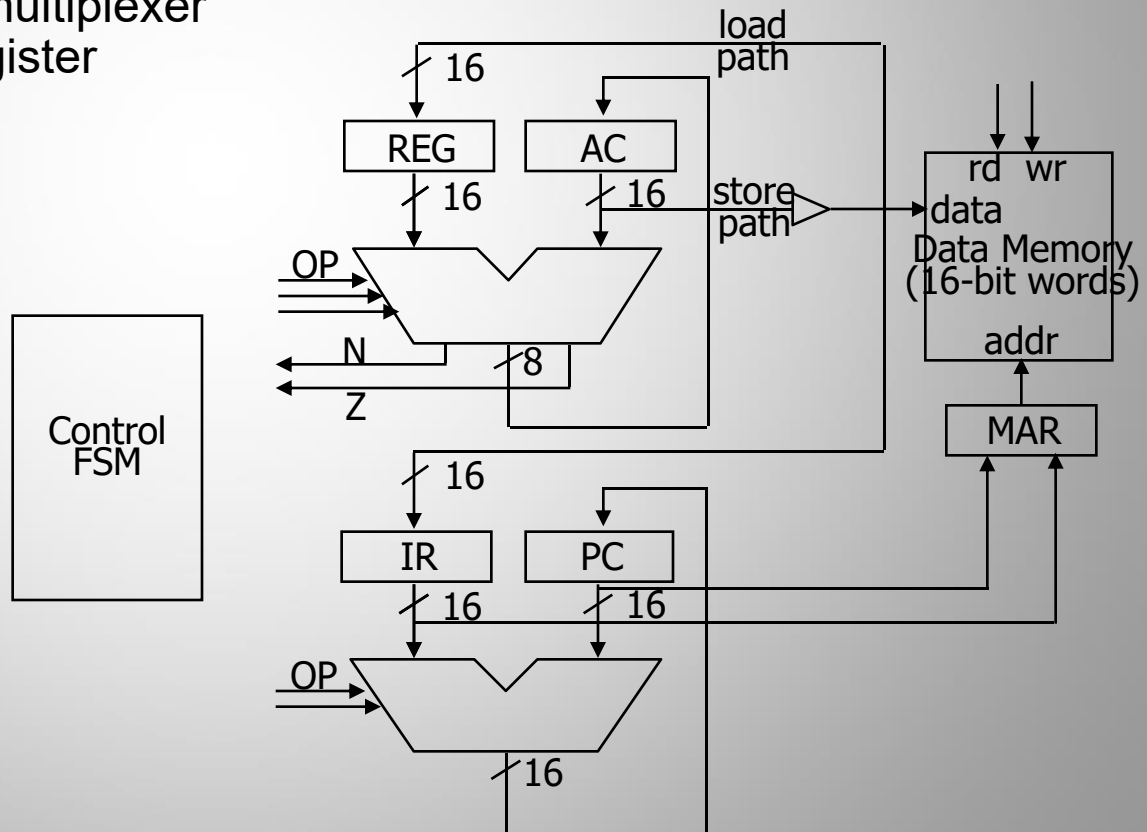
- (3)执行指令

- 每一步将在有限状态机的一个状态被执行。
 - 每个指令通常在一个时钟周期内执行
 - 有些情况下一些存储器操作的指令可能需要两个甚至更长的时钟周期, 这种情况下就需要更多的状态来实现正确的时序。

- 一般而言, 程序的指令都是存储在外部寄存器中的, 所以除了CPU外, 微处理器一般还需要外部存储器以及连接外部存储器与CPU的地址以及数据总线。

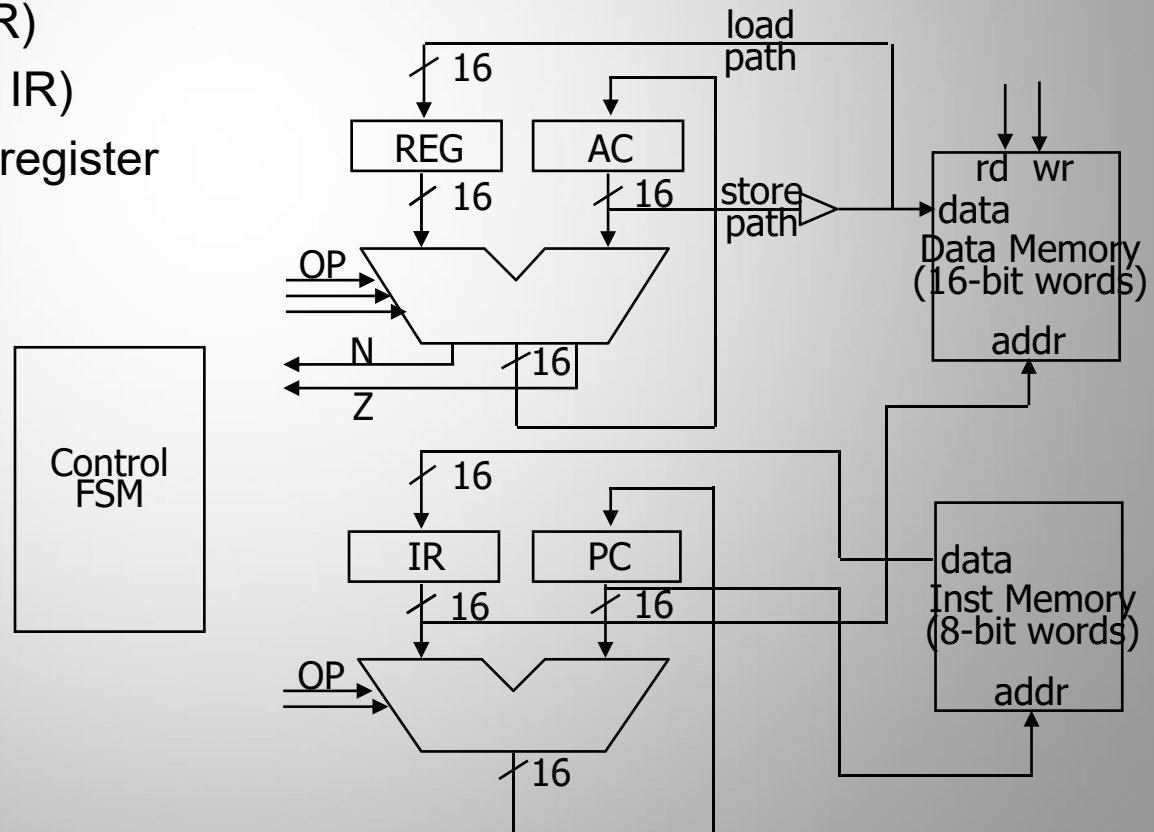
Block Diagram of Processor

- Register Transfer View of **Princeton** Architecture
 - Which register outputs are connected to which register inputs
 - Arrows represent data-flow, other are control signals from control FSM
 - MAR may be a simple multiplexer rather than separate register
 - MBR is split in two (REG and IR)
 - Load control for each register



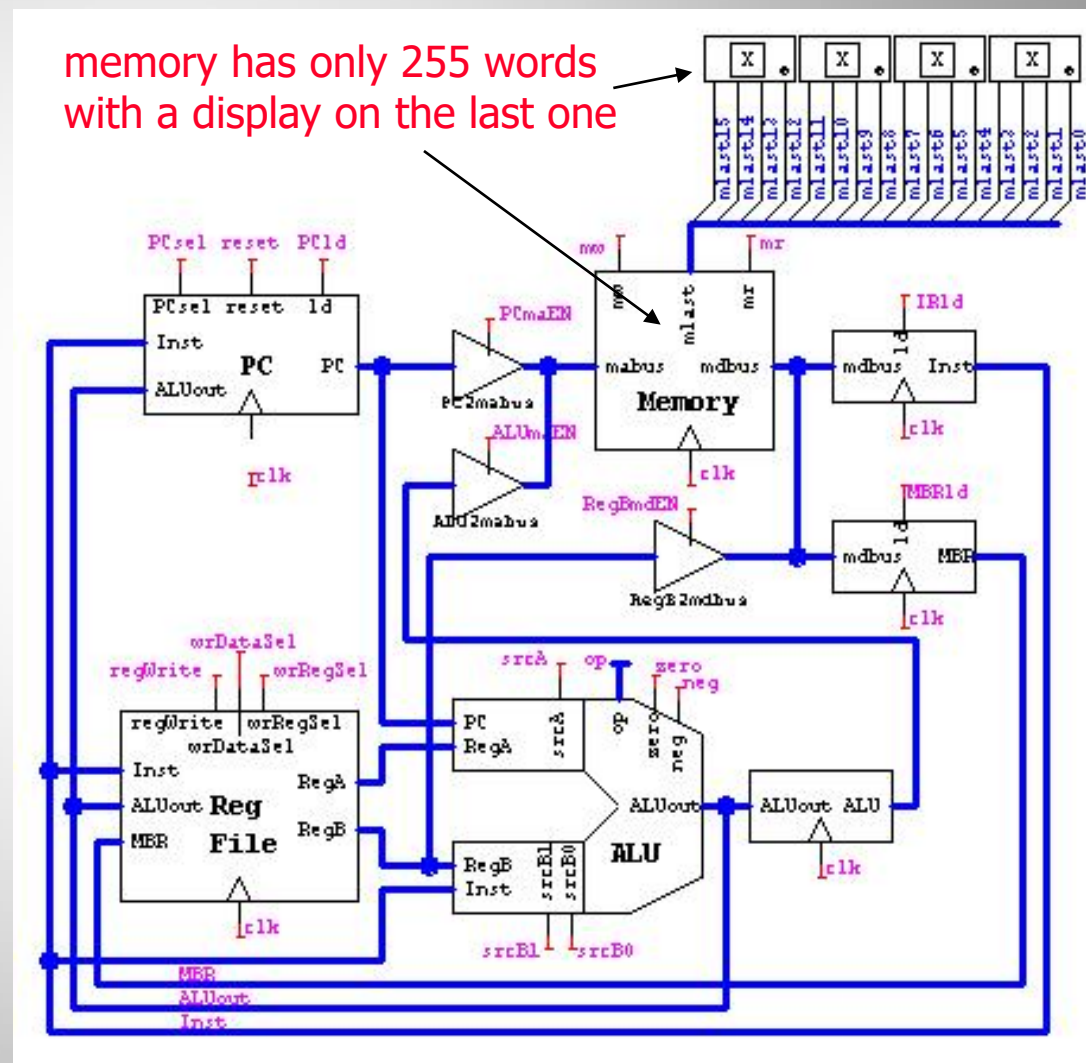
Block Diagram of Processor

- Register transfer view of **Harvard** architecture
 - Which register outputs are connected to which register inputs
 - Arrows represent data-flow, other are control signals from control FSM
 - Two MARs (PC and IR)
 - Two MBRs (REG and IR)
 - Load control for each register

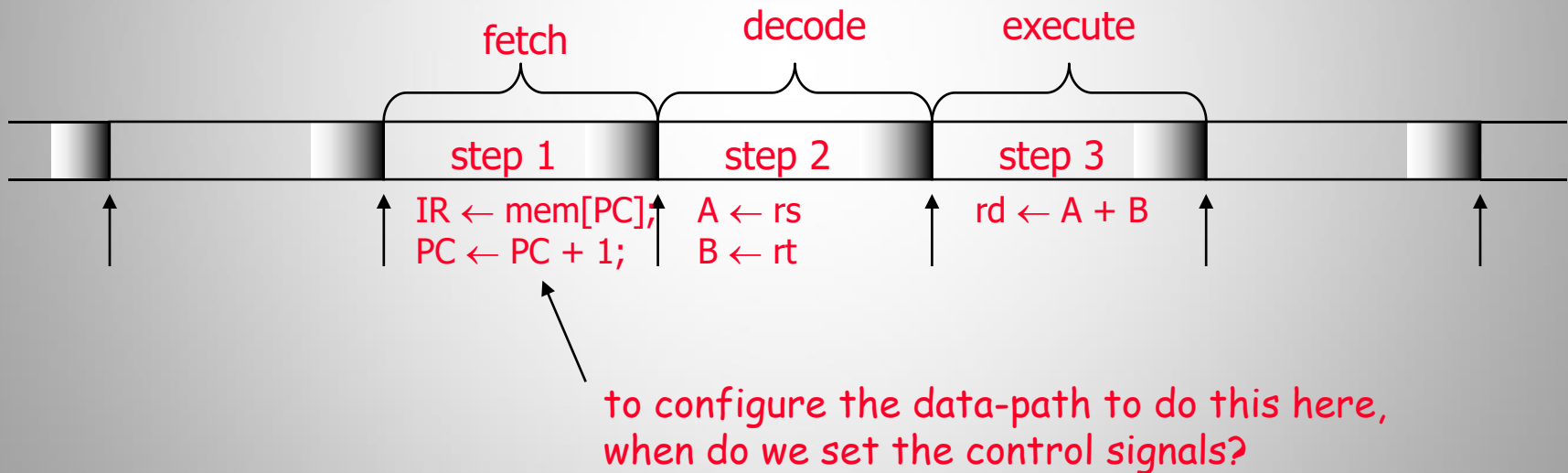


A Simplified Processor Data-path and Memory

- Princeton architecture
- Register file
- Instruction register
- PC incremented through ALU
- Modeled after MIPS rt000 (used in 61C textbook by Patterson & Hennessy)
 - Really a 32 bit machine
 - We'll do a 16 bit version



Review of FSM Timing



FSM Controller for CPU

- Now we need to repeat this for all the instructions of our processor
 - Fetch and decode states stay the same
 - Different execution states for each instruction
 - » Some may require multiple states if available register transfer paths require sequencing of steps

10.5 微码型微处理器设计

微码处理器主要特征

- 控制器CU
- ----微码控制器
 - 存储器+微代码(Micro Code)方式实现

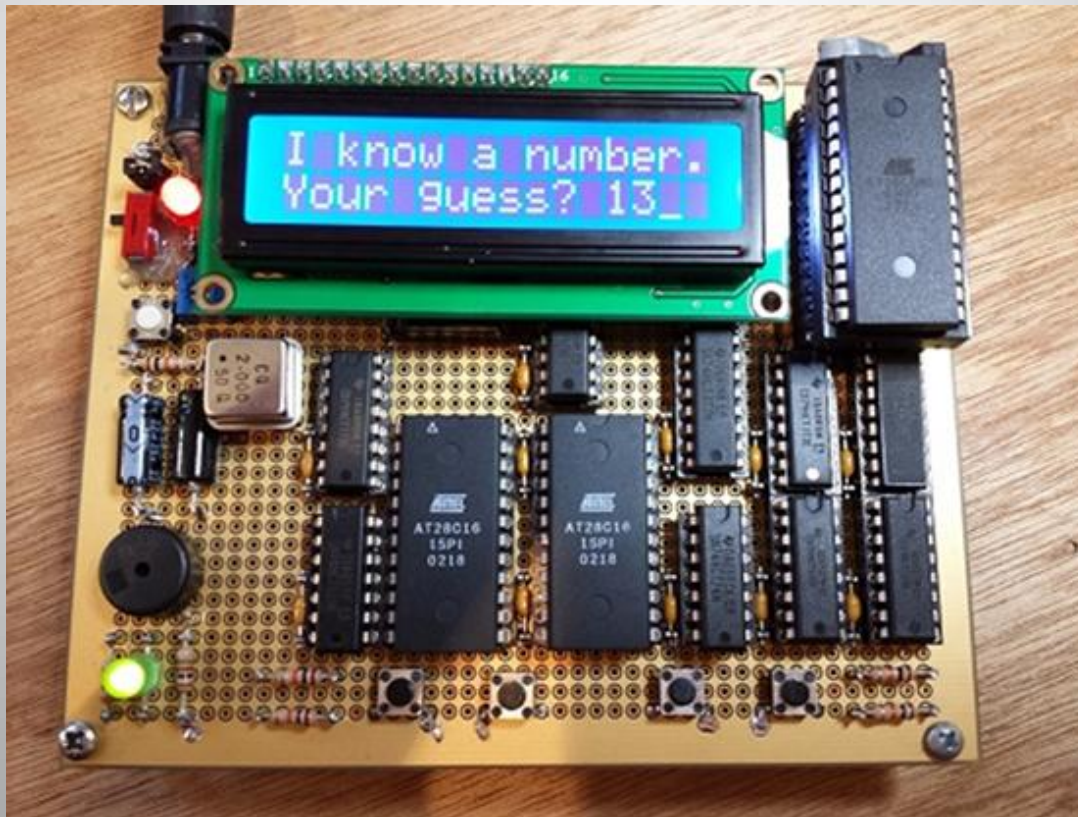
微处理器设计思路

- 1) 总线设计
- 2) 寄存器设计
 - A寄存器、T寄存器、B寄存器
 - IR寄存器、PC寄存器、MA寄存器
- 3) **ALU运算器**
 - 74HC/LS181
- 4) **控制器CU**
 - 硬布线(Hard Wired)方式
 - 微代码(Micro Code)方式
- 5) **I/O部分设计**
- 6) **存储器部分**
 - 存储架构
 - » 冯诺依曼架构
 - » 哈佛架构。

实例学习---Nibbler 4 Bit CPU（开源）

• 1) Summary概要

- Nibbler是一个4 位的CPU
- 用标准7400系列逻辑芯片组建
- 包括计数器、寄存器、缓冲器和门电路等
- 总共只有13个芯片
- 5 x 4 英寸(127 x 102 mm)大小的试验电路板

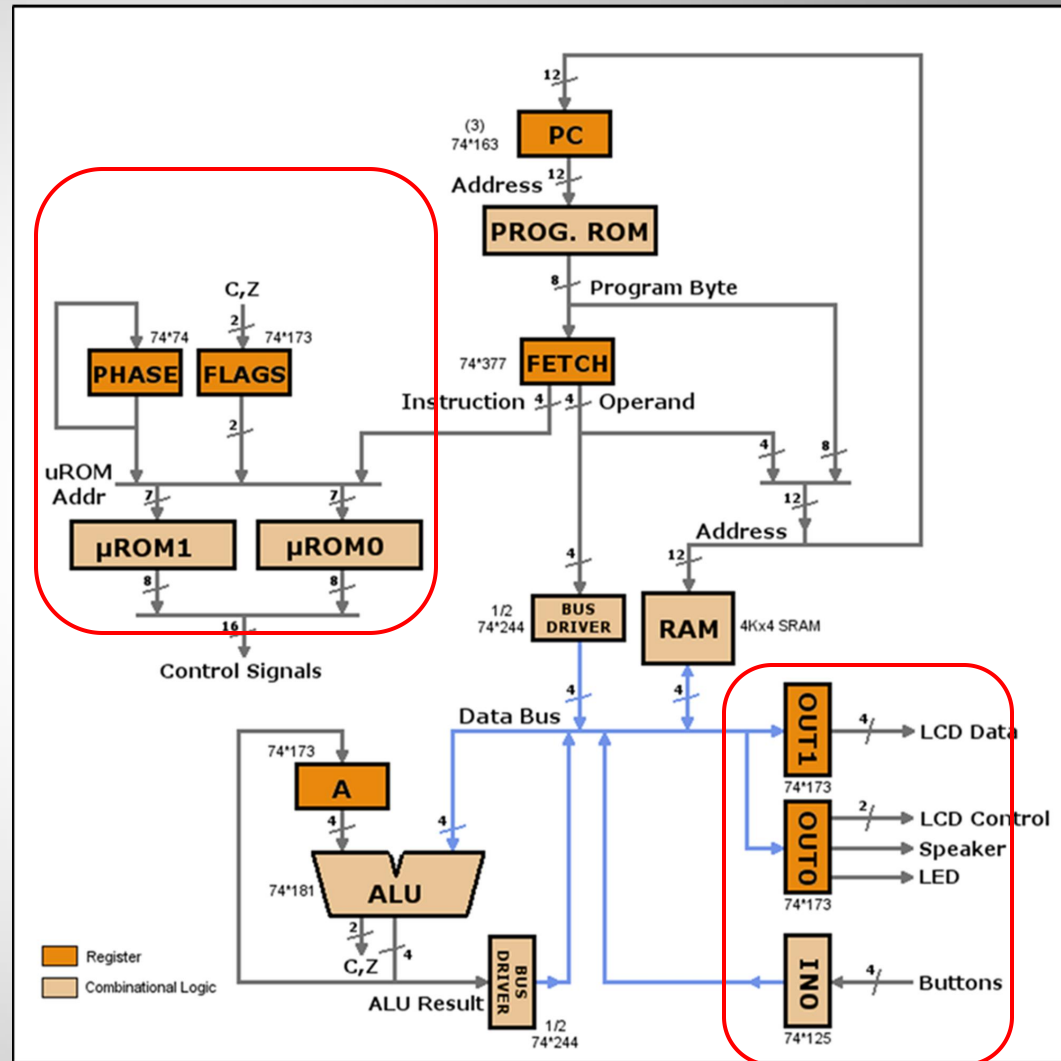


Nibbler

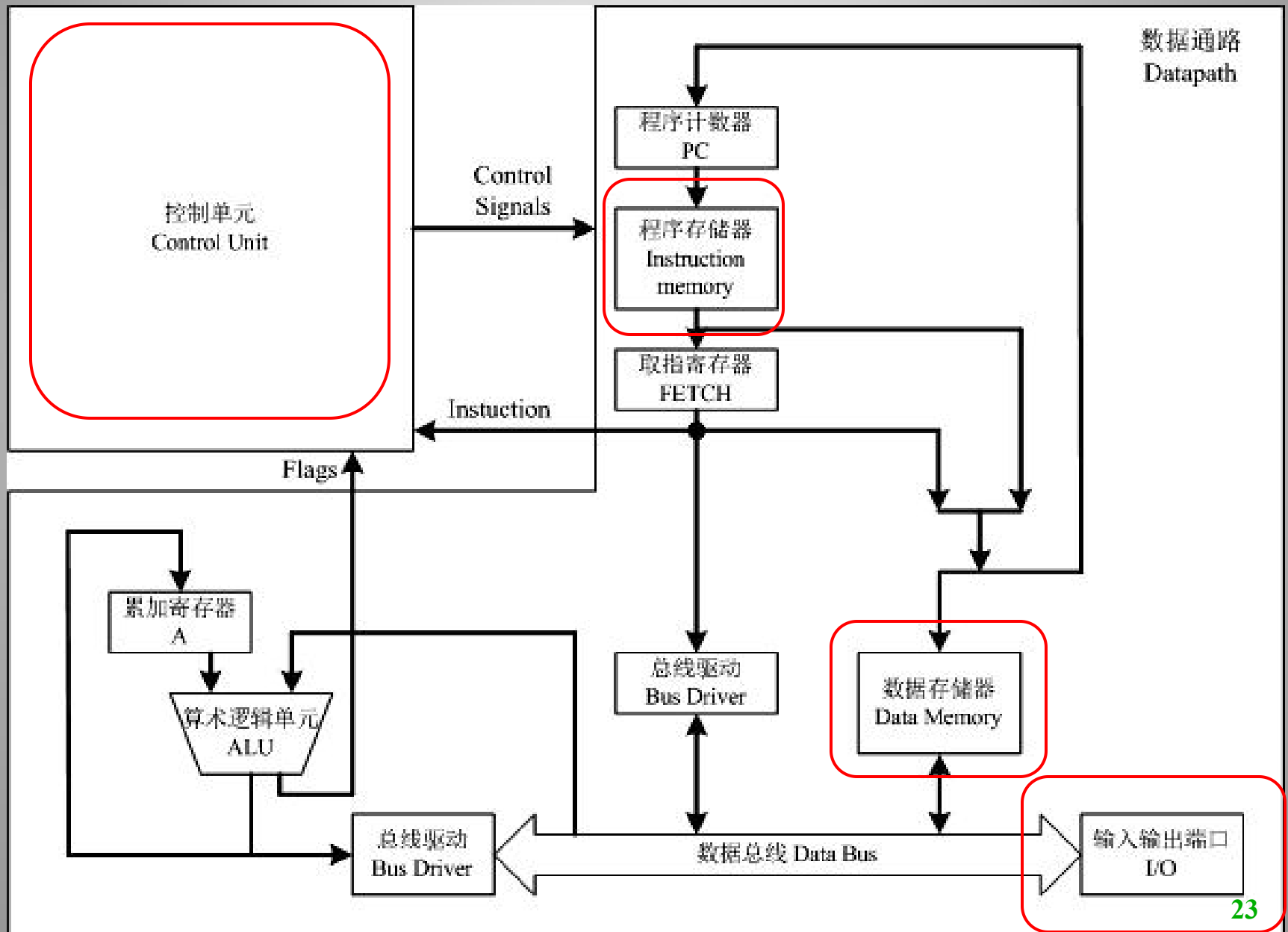
- 2) Specs规格说明
 - 4 位定制CPU
 - 12位寻址, 4K地址空间大小
 - 哈佛体系结构—独立的程序和数据存储空间
 - 2.46 MHz时钟速度, 每条指令2个时钟=123万个指令/秒
 - 在CPU数据和控制通道中使用13个7400系列芯片
 - 2个28C16 EEPROMS用于存储产生内部控制信号的微代码
 - 4K x 4 SRAM (CY7C168A)
 - 28C64 EEPROM 存储程序
 - 4个按键输入
 - 16 x 2字符LCD显示器
 - 1个音频扬声器
 - 5V电源, 大约100 mA电流

3) Architecture体系结构-1

- 程序ROM的地址
- 程序计数器(PC)
- 微码ROM地址
- 程序字节与其他状态位
- 内部控制信号
- 置数(load)、使能(enable)和递增(increment)
- 16个总控制信号, 每个ROM输出8个
- ROM在适当的时间生成
- 程序字节(program byte)
- 构造RAM和跳转(jump)目标的地址
- 将立即数放在数据总线上

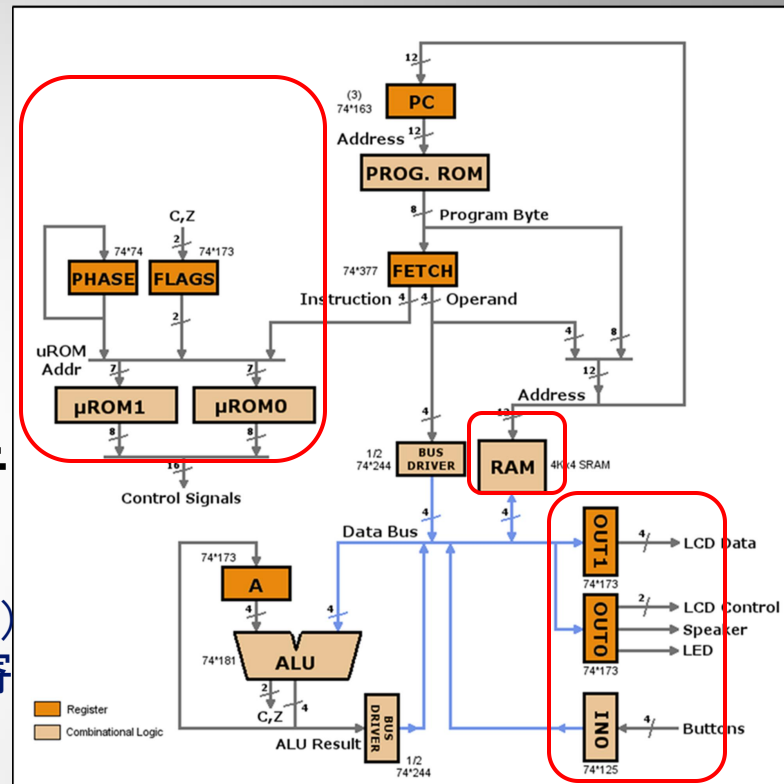


Nibbler结构框图



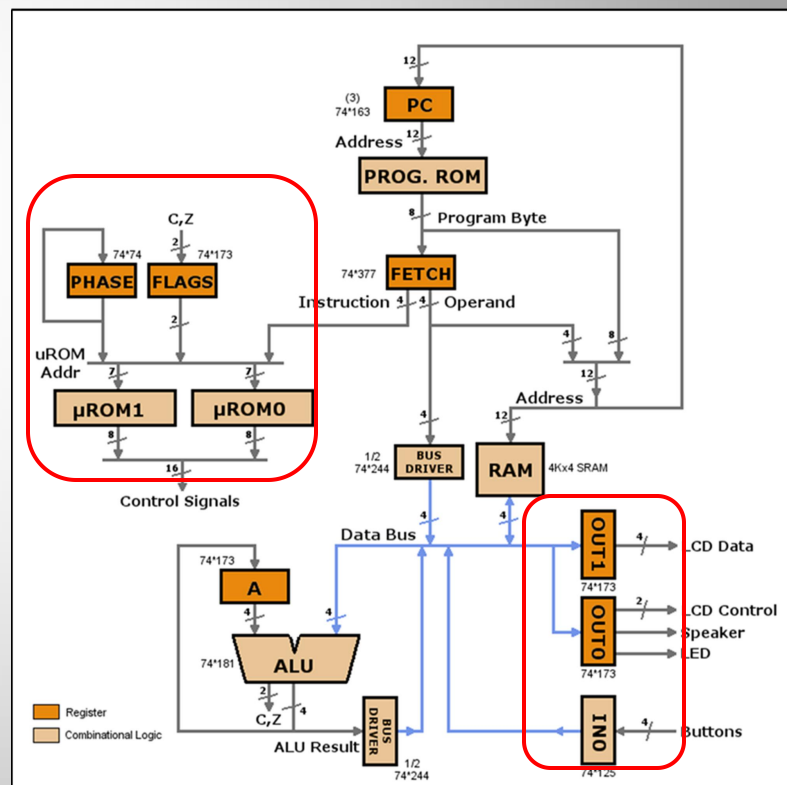
3) Architecture体系结构-2

- 数据总线是**4位**宽
- 程序总线为**8位**宽
 - 4位的指令操作码(opcode)
 - 4位的直接操作数或目标地址的高位
- 指令操作码是**4位**宽
 - 有16种可能的指令类型
- 所有指令都需要两个时钟周期来执行
 - 第一个时钟周期, 也称为**0相**(phase 0)
 - » 指令操作码(opcode)和操作数(operand)从存储器中取出,并存储在取指(Fetch)寄存器中
 - 在第二时钟周期, 称为**1相**(phase 1)
 - » 执行指令所需的计算或操作
- 使用**累加器**load-store设计
 - 所有计算结果存储到4位累加器寄存器“A”中
 - 数据可以在“A”和RAM中的内存位置之间移动
 - 否则所有CPU指令只在“A”上运行



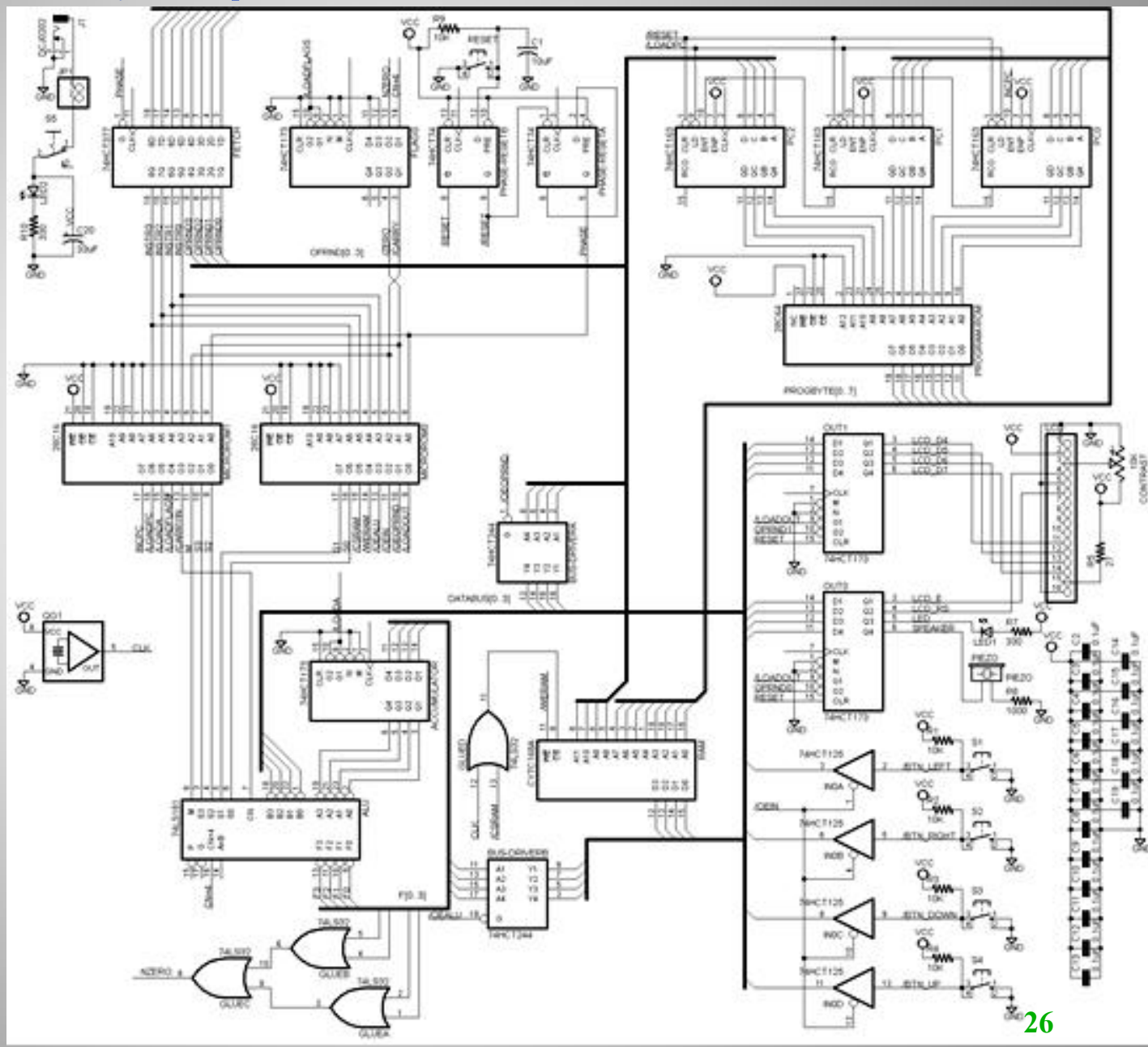
3) Architecture体系结构-3

- **程序和数据地址都是12位宽**
 - 总可寻址存储空间为
 - » 程序4096个8位的字节
 - » 数据的4096个4位的半字节
 - **指令操作码是4位宽**
 - » 涉及绝对内存地址的指令是 $4+12=16$ 位的大小, 或者是两个程序字节
 - » 仅涉及4位立即数的指令大小为 $4+4=8$ 位, 或一个程序字节。
- **支持16个输入和16个输出的I/O设备**
 - Nibbler只使用了1个IN端口和2个OUT端口
 - **输入来自4个瞬时按键**
 - » 分为上/下/左/右配置
 - » 连接到单独的IN端口
 - **输出利用两个OUT端口**
 - » 包括1个用于调试的编程LED
 - » 以及1个用于软件控制声音的压电扬声器
 - » 以及1个 16×2 字符的LCD显示器



4) Schematic原理图

- 除了RAM和ROM之外
- 所有芯片都是常见的7400系列的器件
- 大部分使用了7400HCT逻辑系列的器件
- 功耗更低



5) Limitations局限性

- **没有地址译码器**

- 因为不需要将多个芯片映射到相同地址空间的不同区域
- 程序ROM占用了所有的程序地址空间
- RAM占用了所有的数据地址空间
- I/O外围设备不是内存映射的
- 而是使用指定端口的输入和输出指令来传输数据

- **没有任何地址寄存器**

- 这意味着它不能支持任何形式的间接寻址
- 也不支持硬件控制的堆栈
- 所有内存引用都必须使用绝对地址

- **调用/返回机制**

- 使用跳转表和专用内存位置
- 不需要一个真正的堆栈

6) Program Data程序数据

• 在中间右边是程序ROM

(28C64)

- 存储当前正在运行的程序。
- 这是一个8Kx8的EEPROM
- 但Nibbler的地址大小只允许4K的程序, 所以地址线的A12输入脚未使用, 直接硬连线到0。
- 程序存储器是8位宽的, 因此ROM的所有8个I/O线都被使用。

• 根据指令的类型

- 这些指令可以是4位指令操作码加4位立即操作数;
- 或者4位指令操作码加4位地址, 再跟着8位的地址。
- 在每个指令执行的开始时
 - » 这个程序字节被加载到取指 (Fetch=IR) 寄存器中。

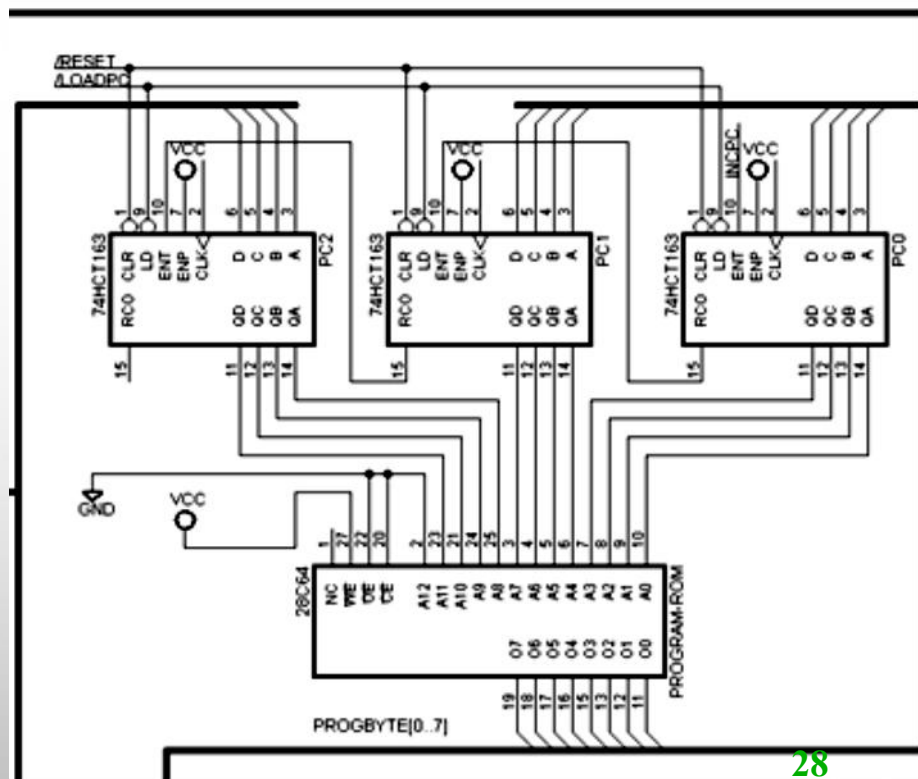
当前执行的程序指令的地址, 存储在程序计数器 (PC) 中

PC由3个4位计数器74HCT163组成, 串接在一起构成12位的逻辑寄存器。

在大多数指令之后, PC将递增指向下一条指令。

对于跳转指令, PC还可以置数为一个新的地址。

该地址来自取指寄存器 (Fetch) 的操作数值 (高4位) 和程序ROM字节 (低8位)。

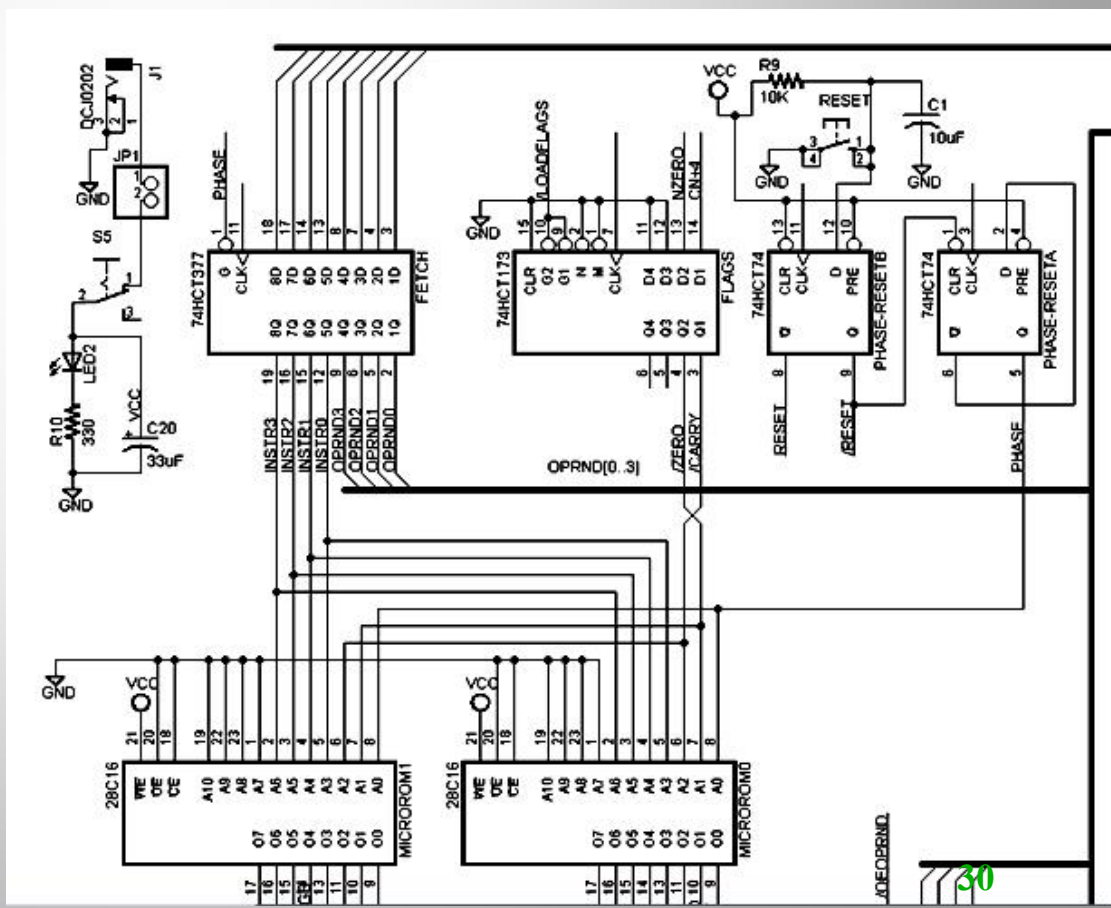


7) Control and Microcode控制与微码

- 左上方与指令执行有关的3个芯片
- 取指寄存器(Fetch)
 - 是一个8位寄存器74HCT377
 - 高4位保存当前指令的操作码, 低4位保存指令或地址的数据
- ALU标志(Flags)
 - 存储在4位标志寄存器74HCT173中
 - 因为只有进位(carry)和零(zero)两个标志, 所以4个比特中的2个未被使用
- 双D触发器74HCT74
 - 一个触发器用于同步复位信号
 - 另一个是相位比特
 - » 指示指令正在执行两个时钟周期中的哪一个
 - » 当相位为0时(Phase 0), 在时钟周期结束时Fetch寄存器加载。

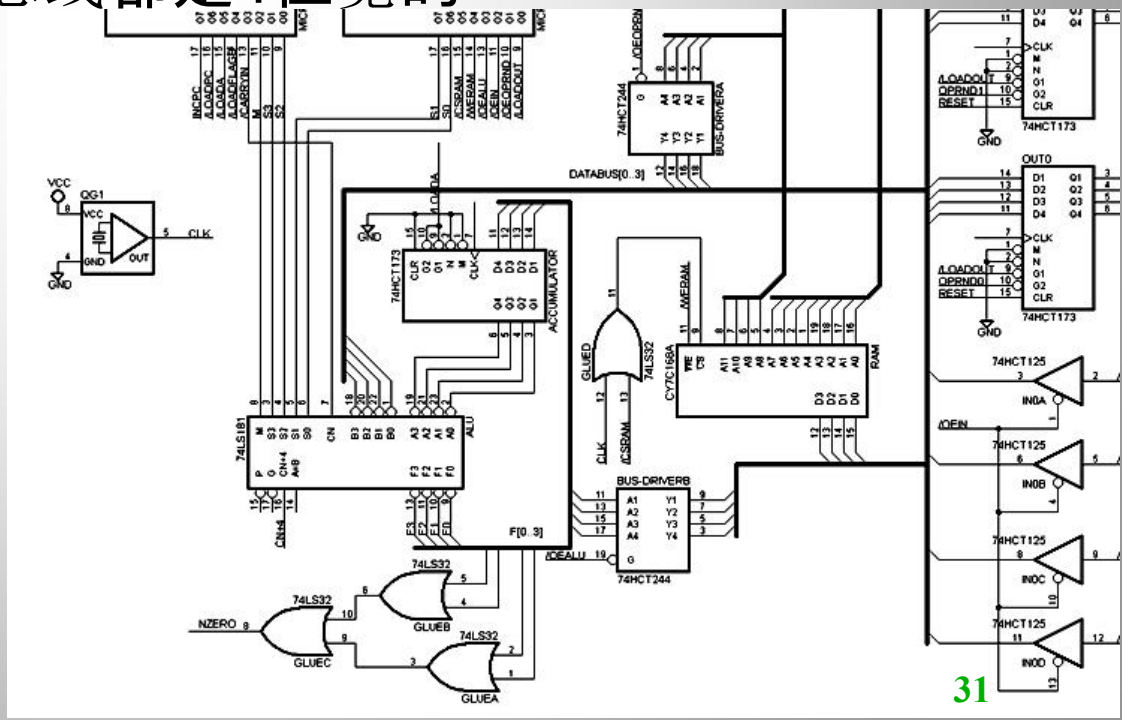
7) Control and Microcode控制与微码

- 指令操作码、ALU标志和相位
 - 组合形成两个微码ROM(28C16)的7位地址, 如图左中部所示
 - 两个ROM的输出构成了协调所有其它芯片行为所需的16个控制信号
 - 微码存储在两个2Kx8 EEPROM中, 所以每个ROM上的十一个地址输入中的四个未被使用, 并且硬连线到0。



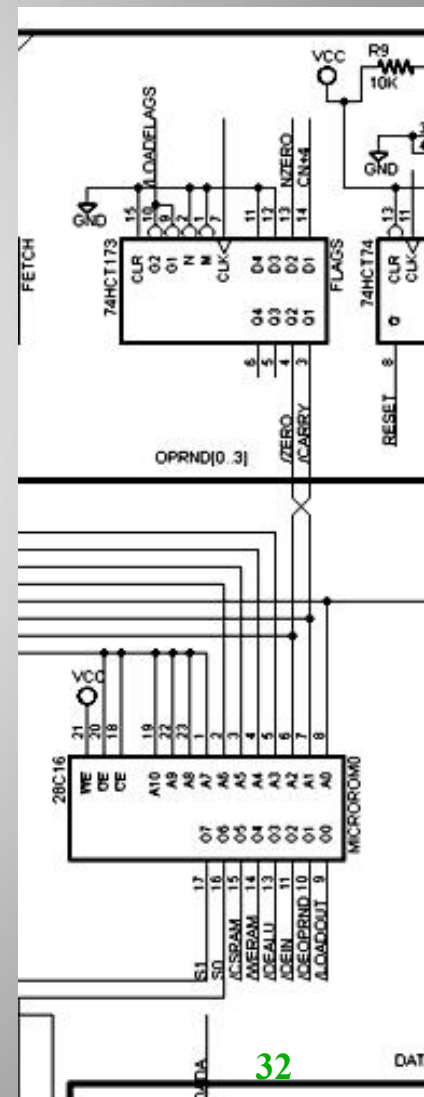
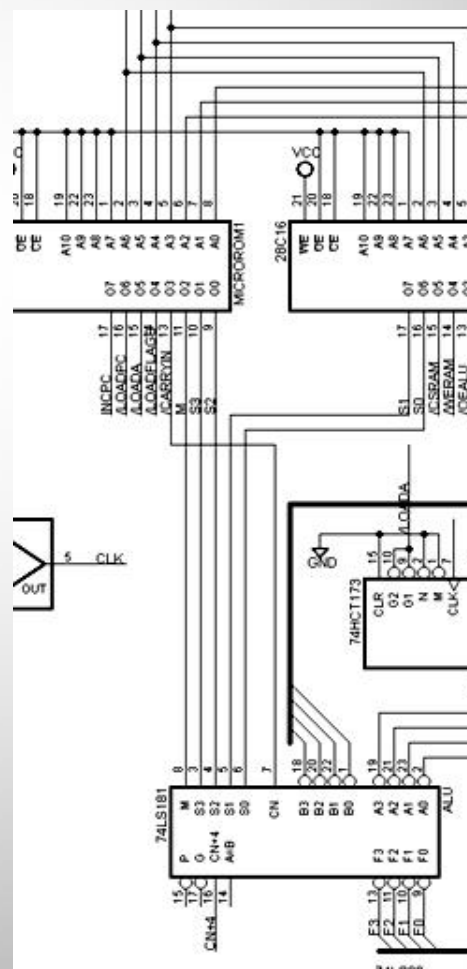
8) ALU Datapath数据通道

- 在原理图的左下方
- 74LS181 ALU(算术和逻辑单元)
 - 可以对其两个输入执行任一常见得算术或逻辑运算
 - 一个输入总是来自累加器, 而另一个输入来自数据总线
 - ALU的计算结果被存储回累加器中
- 累加器寄存器“A”74HCT173
- ALU、累加器和数据总线都是4位宽的



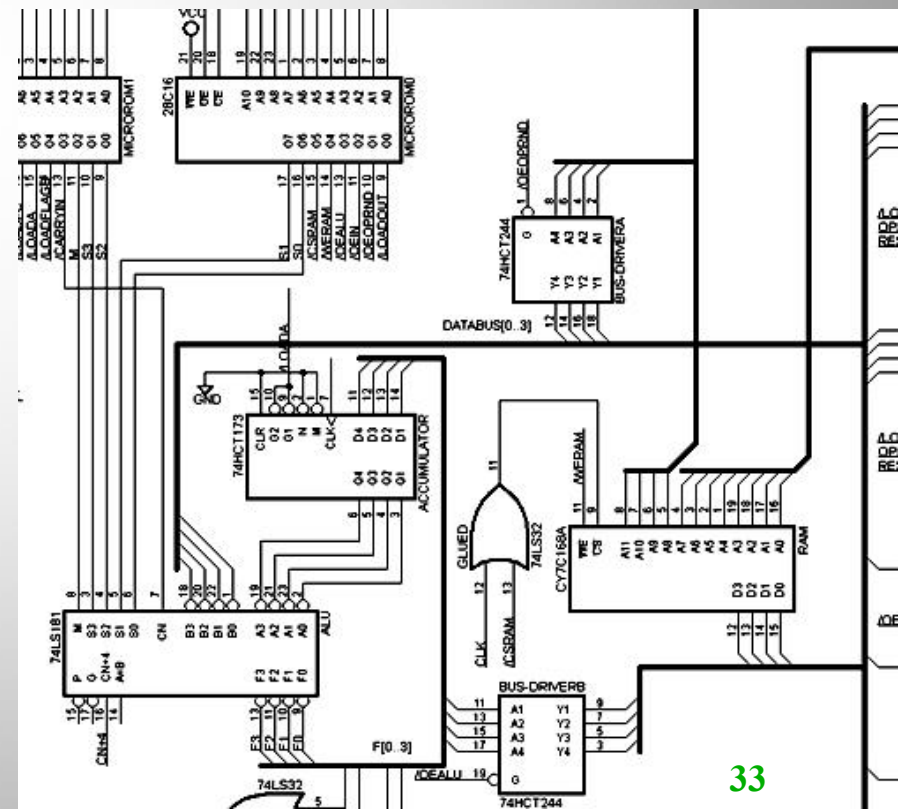
9) Carry-In and Carry Flag进位输入和进位标志

- 进位输入(carry-in bit)
 - 是微码提供的控制信号
- 进位标志(carry flag)寄存器
 - 是算术指令的输出
 - 它不会反馈到ALU中影响后面的计算
 - 程序必须检查进位标志
- 这是一个特意的设计选择
 - 如果进位标志连接到ALU的进位输入比特
 - 那么程序在执行任何单半字节相加之前都需要清除它
 - 进位输入不能简单地硬连接到0
 - 因为CMP(比较)指令需要进位为1, 以便正常工作
 - 所以进位必须由微码提供。



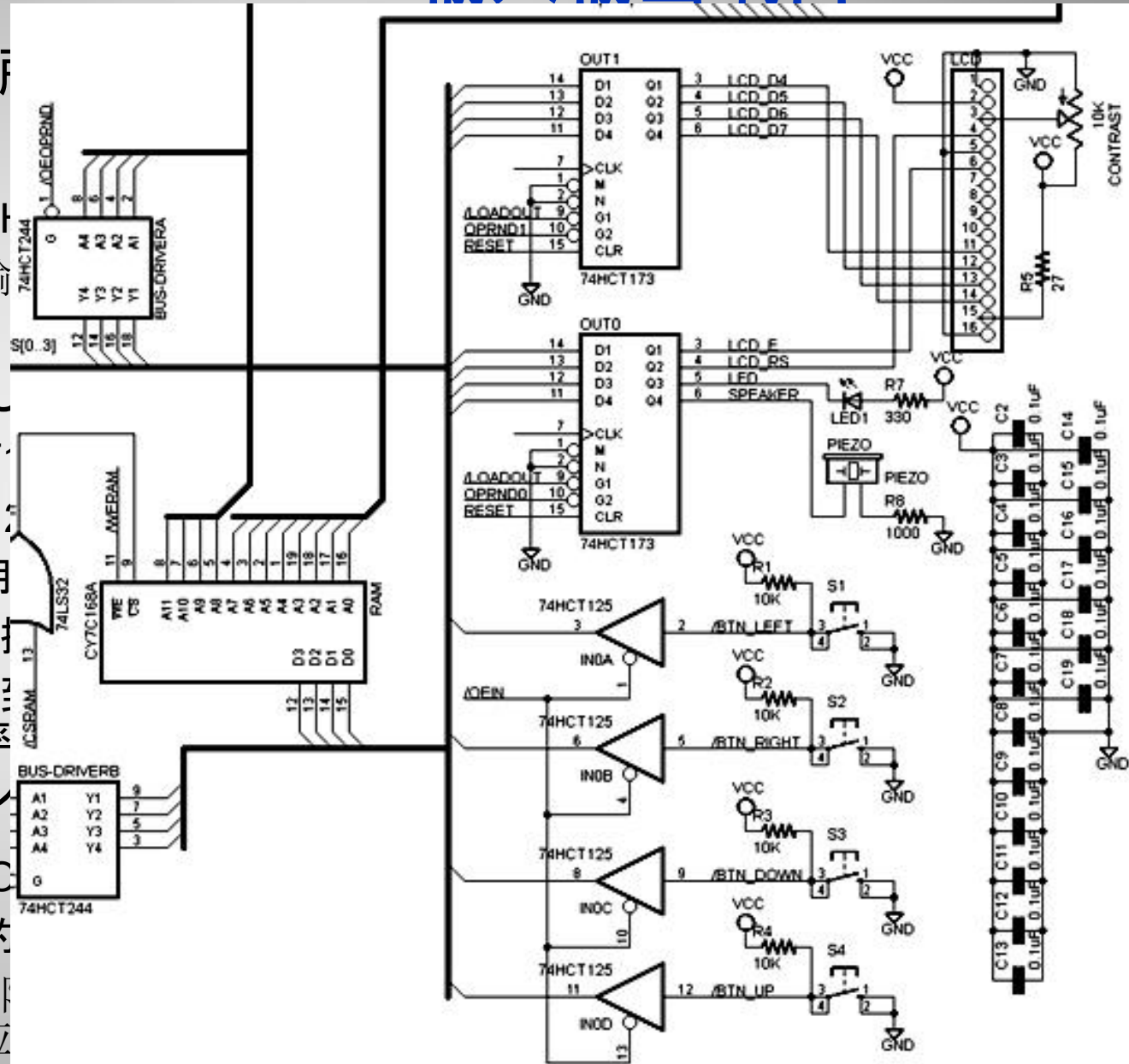
10) RAM随机存取存储器

- 在原理图的底部中间位置
- **RAM芯片CY7C168A**
 - 它的I/O线连接到数据总线
 - 地址来自取指寄存器(Fetch)的操作数(作为高4位)和程序ROM字节(低8位)
 - 这是一个4Kx4 SRAM, 与Nibbler的地址大小和数据宽度匹配
- **RAM的芯片选择输入端/CS**
 - CLK信号与/CSRAM控制信号相或
 - 这意味着RAM将只在每个时钟周期的后半部分使能, 而且只有在微码专门使能它时才被使能
 - 这是当写使能信号/WE和RAM地址可能尚未有效时, 在时钟周期的上半部分防止错误写入RAM的简单方法。



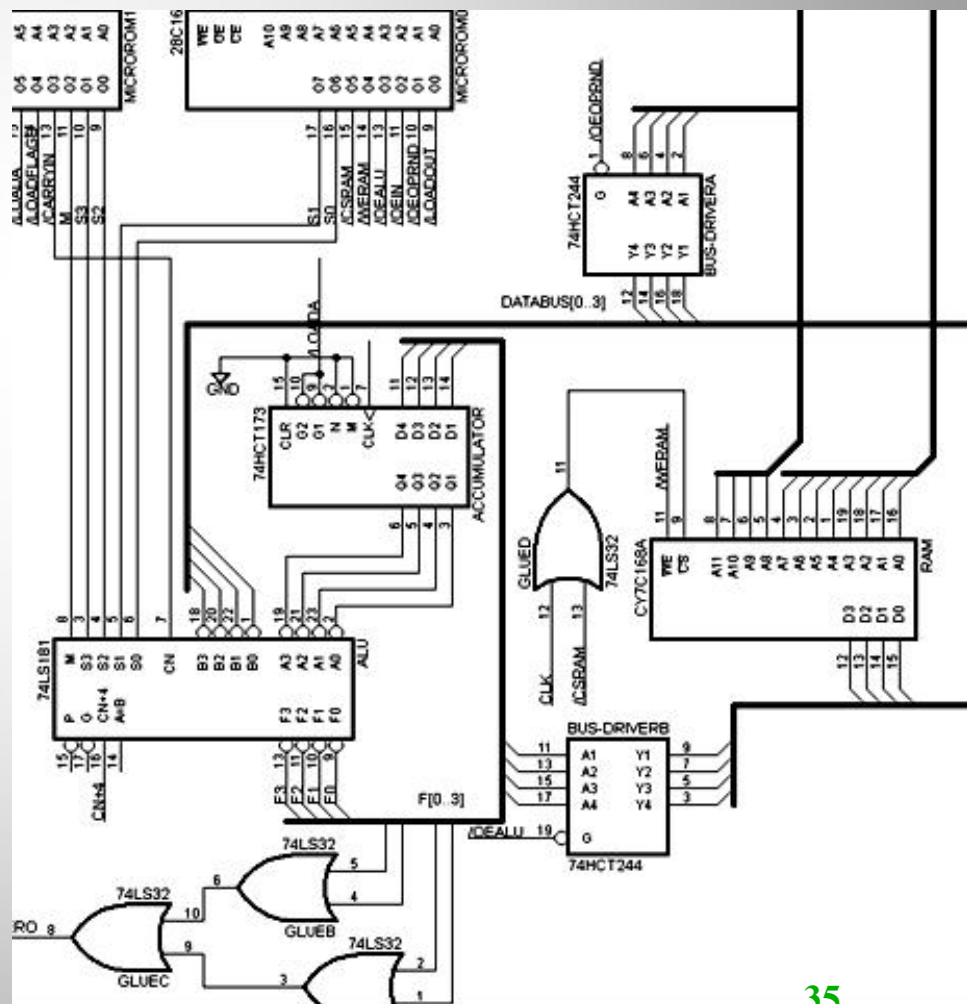
11) IN and OUT Ports 输入输出端口

- 原理图的右下方
- 输入端口IN0
 - 4位总线驱动器74HCT244
 - 因为只有一个IN输入的任何端口号
- 两个输出端口OUT0和OUT1
 - 4位寄存器74HCT173
 - OUT1连接到16×2 LCD显示器
 - 该LCD显示器采用
 - OUT0两条线用于
 - OUT0的一条连接到
 - 脉冲产生不同频率
- 74HCT173有两个
 - G1连接到/LOADC
 - 输出OUT端口号的
 - 这意味着OUT0实
 - 何端口号进行响应



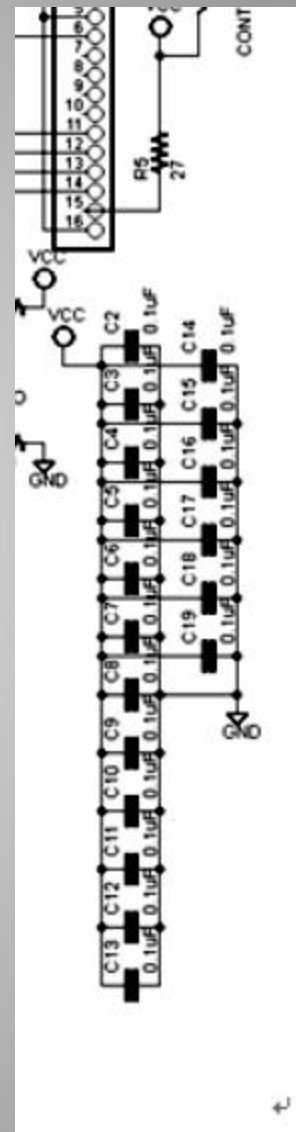
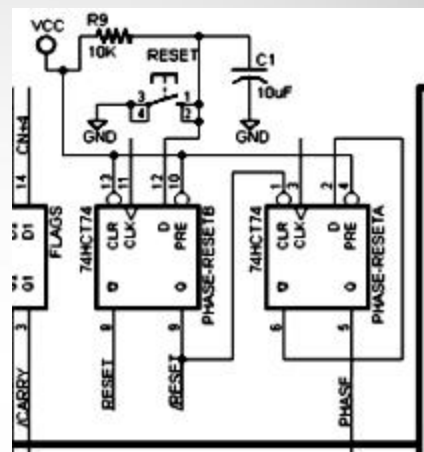
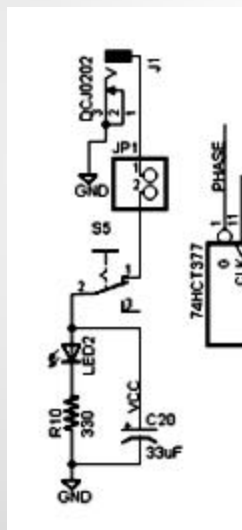
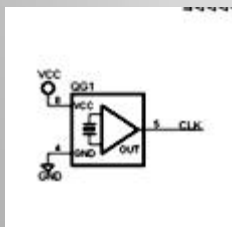
12) Bus Drivers总线驱动

- 在原理图中心底部所示
- 8位驱动74HCT244，一对4位总线驱动器
 - 一半用于将ALU结果驱动到数据总线上，这是将数据存储在RAM或OUT端口时所必需的
 - 另一半用于将操作数从取指寄存器驱动到数据总线上，这对包含立即常数的指令是必要的



硬件电路和辅助元件

- 1) 高频滤波电容(芯片电源)
- 2) 系统复位电路RESET
- 3) 电源输入
- 4) 时钟



13) Combinatorial Feedback Loops组合反馈环路

- **ALU输出通过总线驱动器B，返回到ALU的B输入端**
- 如果ALU功能被设置为类似 $A+B$ ，而总线驱动B使能，那么显然会产生反馈问题，因为ALU的输出也将是它的输入
- 即使ALU功能被设置为A，也可以创建反馈环路，只是输入B上的值是逻辑无关的
- 可以证明逻辑上无关的并不总是意味着电气无关，数据总线上的瞬态无效电压可以将ALU锁定在一个恶性循环中，其中的无效输入会导致无效输出，进一步导致更多无效输入
- 反馈环路问题可在微码中修复，使用0000输出替代原来的ALU功能选择1111即可
- 微码为基础的修复可能不起作用。如果正碰到这个问题，在数据总线和ALU的B输入之间可以放置一个施密特触发缓冲器74HCT7541。

14) Microcode and Instruction Set 微码和指令集

- 两种类型指令

- 立即数指令和寻址指令
- 都以4位指令操作码开始
- ADDI立即指令包括嵌入在指令中的4位操作数值，指令总共有8位。
- 寻址指令JMP包括在数据空间(RAM)或程序空间(ROM)的12位地址，指令长度总共有16位。

- 指令编码非常简单

type	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
addressed	i3	i2	i1	i0	a11	a10	a9	a8	a7	a6	a5	a4	a3	a2	a1	a0
immediate	i3	i2	i1	i0	o3	o2	o1	o0								

- 4比特指令操作码*i*[0..3]与ALU进位标志C和零标志Z以及相位比特相结合，以形成两个微码ROM的7位地址
- ROM的输出所需的16个控制信号

控制信号(MicrocodeROM输出)

- 1) incPC= ENT@74HCT163(PC)
- 2) /loadPC=/LD@74HCT163(PC)
- 3) /loadA=/oe1,/oe2@74HCT173(A)
- 4) /loadFlags=/oe1,/oe2@74HCT173(FLAGS)
- 5) /carryIn=CIN@74LS181(ALU)
- 6-10) M S3,S2,S1,S0=@74LS181(ALU模式控制)
- 11) /csRAM= CLK相或/CSRAM=/cs@ CY7C168A(RAM)
- 12) /weRAM= /we@ CY7C168A(RAM)
- 13) /oeALU= /oe@74HCT244(ALU到DATA总线驱动)
- 14) /oeIN=/INOA~/INOD@74HCT125(输入驱动三态控制)
- 15) /oeOpend= /oe@74HCT244(FETCH到DATA总线驱动)
- 16) /loadOut=/oe1@74HCT173(输出寄存器)

74HCT173=三态输出的四路 D 触发器, When either OE1 or OE2 is high, the output is disabled to the high-impedance state.

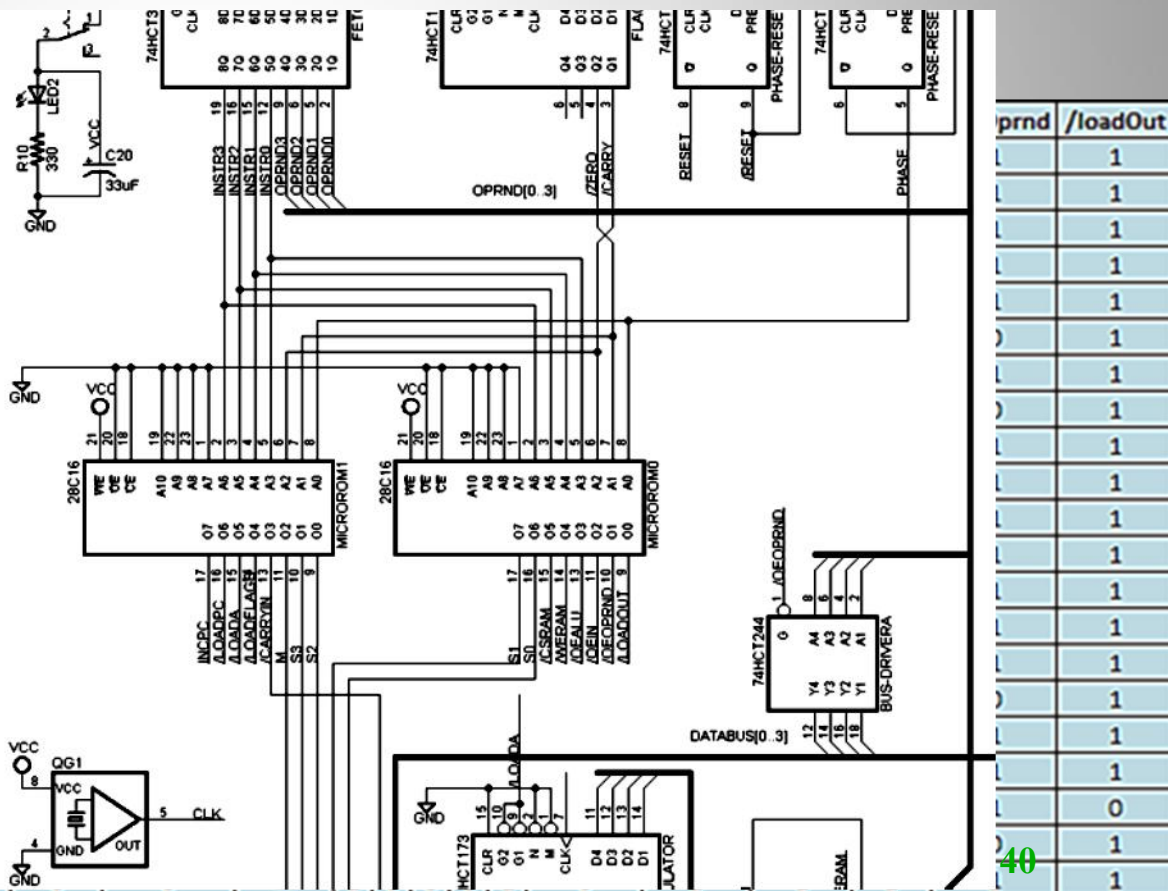
微码ROM

- 所有指令在第0相是相同的=表的第一行phase0

- 这时指令还没从内存中读取, CPU甚至不知道它执行什么指令!
- 微码将累加器的内容驱动到数据总线上, 即使它们不需要
- 如果总线浮空CMOS芯片可能会出现问題(所以不允许悬空)
- 在第0相(phase 0), 程序寄存器ROM字节硬件加载到取指寄存器(Fetch)
- 微码将程序计数器增

注: 有一半地址的内容是与第一行相同的phase0

instruction	i3	i2	i1	i0	/C	/Z	phase	incPC	/loadf
any	x	x	x	x	x	x	0	1	1
JC	0	0	0	0	0	x	1	0	0
JC	0	0	0	0	1	x	1	1	1
JNC	0	0	0	1	0	x	1	1	1
JNC	0	0	0	1	1	x	1	0	0
CMPI	0	0	1	0	x	x	1	0	1
CMPM	0	0	1	1	x	x	1	1	1
LIT	0	1	0	0	x	x	1	0	1
IN	0	1	0	1	x	x	1	0	1
LD	0	1	1	0	x	x	1	1	1
ST	0	1	1	1	x	x	1	1	1
JZ	1	0	0	0	x	0	1	0	0
JZ	1	0	0	0	x	1	1	1	1
JNZ	1	0	0	1	x	0	1	1	1
JNZ	1	0	0	1	x	1	1	0	0
ADDI	1	0	1	0	x	x	1	0	1
ADDM	1	0	1	1	x	x	1	1	1
JMP	1	1	0	0	x	x	1	0	0
OUT	1	1	0	1	x	x	1	0	1
NORI	1	1	1	0	x	x	1	0	1
NORM	1	1	1	1	x	x	1	1	1



第1相 (phase 1)

• ADDI指令(8bit)

- 它的指令操作码是二进制的1010, 或十六进制的\$A
- 无论C或Z标志如何, 都要使用相同的控制信号
- /oeOprnd将为0, 使能总线驱动器能够将立即操作数驱动到数据总线上, 并连接到ALU的一个输入
- ALU的另一个输入总是与累加器寄存器相连
- /carryIn、M、和S[0..3]控制信号将ALU设置成为加法模式中, 不带进位输入
- /loadA将是0, 因此ALU结果将被存储回到累加器
- /loadFlags也将是0, 所以进位和零点标志将按加法的结果更新。

instruction	i3	i2	i1	i0	/C	/Z	phase	incPC	/loadPC	/loadA	/loadFlags	/carryIn	M	S3	S2	S1	S0	/csRAM	/weRAM	/oeALU	/oeIN	/oeOprnd	/loadOut
any	x	x	x	x	x	x	0	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JC	0	0	0	0	0	x	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JC	0	0	0	0	1	x	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNC	0	0	0	1	0	x	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNC	0	0	0	1	1	x	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
CMPI	0	0	1	0	x	x	1	0	1	1	0	0	0	0	1	1	0	1	1	1	1	0	1
CMPM	0	0	1	1	x	x	1	1	1	1	0	0	0	0	1	1	0	0	1	1	1	1	1
LIT	0	1	0	0	x	x	1	0	1	0	0	dc	1	1	0	1	0	1	1	1	1	0	1
IN	0	1	0	1	x	x	1	0	1	0	0	dc	1	1	0	1	0	1	1	1	0	1	1
LD	0	1	1	0	x	x	1	1	1	0	0	dc	1	1	0	1	0	0	1	1	1	1	1
ST	0	1	1	1	x	x	1	1	1	1	1	1	0	0	0	0	0	0	0	0	1	1	1
JZ	1	0	0	0	x	0	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JZ	1	0	0	0	x	1	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNZ	1	0	0	1	x	0	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNZ	1	0	0	1	x	1	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
ADDI	1	0	1	0	x	x	1	0	1	0	0	1	0	1	0	0	1	1	1	1	1	0	1
ADDM	1	0	1	1	x	x	1	1	1	0	0	1	0	1	0	0	1	0	1	1	1	1	1
JMP	1	1	0	0	x	x	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
OUT	1	1	0	1	x	x	1	0	1	1	1	1	0	0	0	0	0	1	1	0	1	1	0
NORI	1	1	1	0	x	x	1	0	1	0	0	dc	1	0	0	0	1	1	1	1	1	0	1
NORM	1	1	1	1	x	x	1	1	1	0	0	dc	1	0	0	0	1	1	1	1	1	1	1

第1相 (phase 1)

- ADDM指令(16bit=FETCH+(PC+1)指向的内容)

- 它的指令操作码是二进制的1011, 或十六进制的\$B
- 无论C或Z标志如何, 都要使用相同的控制信号
- /csRAM为0, RAM地址12bit, 数据只在时钟的后半部分使能, ALU的一个输入
- ALU的另一个输入总是与累加器寄存器相连
- /carryIn、M、和S[0..3]控制信号将ALU设置成为加法模式中, 不带进位输入
- /loadA将是0, 因此ALU结果将被存储回到累加器
- /loadFlags也将是0, 所以进位和零点标志将按加法的结果更新
- incPC程序计数器再加1, 指向下一条指令(跳过地址低8bit)。

instruction	i3	i2	i1	i0	/C	/Z	phase	incPC	/loadPC	/loadA	/loadFlags	/carryIn	M	S3	S2	S1	S0	/csRAM	/weRAM	/oeALU	/oeIN	/oeOprnd	/loadOut
any	x	x	x	x	x	x	0	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JC	0	0	0	0	0	x	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JC	0	0	0	0	1	x	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNC	0	0	0	1	0	x	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNC	0	0	0	1	1	x	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
CMPI	0	0	1	0	x	x	1	0	1	1	0	0	0	0	1	1	0	1	1	1	1	0	1
CMPM	0	0	1	1	x	x	1	1	1	1	0	0	0	0	1	1	0	0	1	1	1	1	1
LIT	0	1	0	0	x	x	1	0	1	0	0	dc	1	1	0	1	0	1	1	1	1	0	1
IN	0	1	0	1	x	x	1	0	1	0	0	dc	1	1	0	1	0	1	1	1	0	1	1
LD	0	1	1	0	x	x	1	1	1	0	0	dc	1	1	0	1	0	0	1	1	1	1	1
ST	0	1	1	1	x	x	1	1	1	1	1	1	0	0	0	0	0	0	0	0	1	1	1
JZ	1	0	0	0	x	0	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JZ	1	0	0	0	x	1	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNZ	1	0	0	1	x	0	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNZ	1	0	0	1	x	1	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
ADDI	1	0	1	0	x	x	1	0	1	0	0	1	0	1	0	0	1	1	1	1	1	0	1
ADDM	1	0	1	1	x	x	1	1	1	0	0	1	0	1	0	0	1	0	1	1	1	1	1
JMP	1	1	0	0	x	x	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
OUT	1	1	0	1	x	x	1	0	1	1	1	1	0	0	0	0	0	1	1	0	1	1	0
NORI	1	1	1	0	x	x	1	0	1	0	0	dc	1	0	0	0	1	1	1	1	1	0	1
NORM	1	1	1	1	x	x	1	1	1	0	0	dc	1	0	0	0	1	1	1	1	1	0	1

第1相 (phase 1)

• JZ(零则跳转)指令(16bit)

- 指令操作码是二进制的1000, 或十六进制的\$8
- 如果Z标志是0(意思是true), 那么incPC将是0(PC不会递增), /loadPC将是0(PC将被加载新地址)。如果Z是0将跳转
- 如果Z标志为1(意味着false), 那么incPC将是1(PC将递增), /loadPC将是1(PC将不置数)。如果Z标志为1, 前进到下一个指令, 而不需要跳转。
- 跳转指令不涉及ALU, 但将累加器值驱动到数据总线上, 因此总线不会浮空。

instruction	i3	i2	i1	i0	/C	/Z	phase	incPC	/loadPC	/loadA	/loadFlags	/carryIn	M	S3	S2	S1	S0	/csRAM	/weRAM	/oeALU	/oeIN	/oeOprnd	/loadOut
any	x	x	x	x	x	x	0	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JC	0	0	0	0	0	x	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JC	0	0	0	0	1	x	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNC	0	0	0	1	0	x	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNC	0	0	0	1	1	x	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
CMPI	0	0	1	0	x	x	1	0	1	1	0	0	0	0	1	1	0	1	1	1	1	0	1
CMPM	0	0	1	1	x	x	1	1	1	1	0	0	0	0	1	1	0	0	1	1	1	1	1
LIT	0	1	0	0	x	x	1	0	1	0	0	dc	1	1	0	1	0	1	1	1	1	0	1
IN	0	1	0	1	x	x	1	0	1	0	0	dc	1	1	0	1	0	1	1	1	0	1	1
LD	0	1	1	0	x	x	1	1	1	0	0	dc	1	1	0	1	0	0	1	1	1	1	1
ST	0	1	1	1	x	x	1	1	1	1	1	1	0	0	0	0	0	0	0	0	1	1	1
JZ	1	0	0	0	x	0	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JZ	1	0	0	0	x	1	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNZ	1	0	0	1	x	0	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNZ	1	0	0	1	x	1	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
ADDI	1	0	1	0	x	x	1	0	1	0	0	1	0	1	0	0	1	1	1	1	1	0	1
ADDM	1	0	1	1	x	x	1	1	1	0	0	1	0	1	0	0	1	0	1	1	1	1	1
JMP	1	1	0	0	x	x	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
OUT	1	1	0	1	x	x	1	0	1	1	1	1	0	0	0	0	0	1	1	0	1	1	0
NORI	1	1	1	0	x	x	1	0	1	0	0	dc	1	0	0	0	1	1	1	1	1	0	1
NORM	1	1	1	1	x	x	1	1	1	0	0	dc	1	0	0	0	1	0	1	1	1	1	1

15) Instruction Set指令集

- 4位的指令操作码, 16个不同的指令
- 如跳转指令有五种类型:
 - **JMP** – 无条件转移
 - **JC** – 如果进位则跳转
 - **JNC** – 如果无进位则跳转
 - **JZ** – 如果零则跳转
 - **JNZ** – 如果非零则跳转
- 但JNC和JNZ的负跳转不是绝对必要, 可以用正跳和无条件跳转来重写

```
JNC noCarry
carry:
    do stuff
noCarry:
    do other stuff
可以变为
JC carry
JMP noCarry
carry:
    do stuff
noCarry:
    do other stuff
```

15) Instruction Set指令集

- ***LD***和***ST***指令 (16bit)
 - 从数据空间(RAM)加载半字节到累加器, 或者从累加器存半字节到数据空间。
- ***LIT***指令 (8bit)
 - 加载一个立即数半字节值到累加器。它也可以被命名为***LDI***。
- ***OUT***指令 (8bit)
 - 从累加器中存储半字节到16个输出端口中的一个
- ***IN***指令 (8bit)
 - 从16个输入端口中加载半字节到累加器中。这些指令用于I/O。
- ***ADDI***指令 (8bit)
 - 加半字节立即数值到累加器中
- 而***ADDM*** (16bit)
 - 将数据空间中的半字节加到累加器中。
 - 在这两种情况下, 加法的进位输入(carry-in)是0, 而进位输出(carry-out)和零标志由加法的结果设置。

15) Instruction Set指令集

- ***CMPI***指令 (8bit)
 - 将半字节立即数与累加器进行比较 (累加器减去数值)
- ***CMPM*** (16bit)
 - 将数据空间中的半字节与累加器进行比较
 - 累加器的值没有被修改, 但进位输出和零标志是由减法的结果决定的
 - 因为ALU正在执行减法, 所以进位输入总是为1, 以便使算术工作正确
- ***NORI***指令 (8bit)
 - 执行与立即数的NOR (或非)
- ***NORM*** (16bit)
 - 执行与数据空间值的NOR
 - 进位输出 (carry-out) 和零标志是由NOR的结果设置的

16) Synthetic Instructions合成指令

- 那些不包括在表里的指令可以根据现有的指令合成
 - 比如减法、与、或、非
- 汇编器程序中包含了一些宏
 - 它们可以用作内置指令

NOT: NORI #0

ORI x: NORI x; NORI #0

ORM x: NORM x; NORI #0

ANDI x: NORI #0; NORI ~x

SUBI: ADDI (~x+1)

- 无法合成的通用指令是**ANDM**和**SUBM**
 - 它们需要使用临时寄存器来保存数据空间值的补码
 - 数据空间中的专用位置可以用于此目的
 - Nibbler只有一个寄存器
- 在许多情况下, 通过一些思考可以完全消除对这些合成指令的需要。

17) Software Tools 软件工具

- (1) Microcode Builder 微代码生成器
 - 微码生成器是最简单的工具
 - 它为两个微代码ROM的内容生成两个二进制文件
 - 直接C代码来生成所需的二进制数据
- 该程序包含大量的比特移位功能

```
// 9 JNZ
    for (int c = 0; c < 2; c++)
    {
        microcode[(9 << 3) | (c << 2) | (0 << 1) | 1]
        = 0xF03F; // Z
        microcode[(9 << 3) | (c << 2) | (1 << 1) | 1]
        = 0x303F; // NZ
    }
```

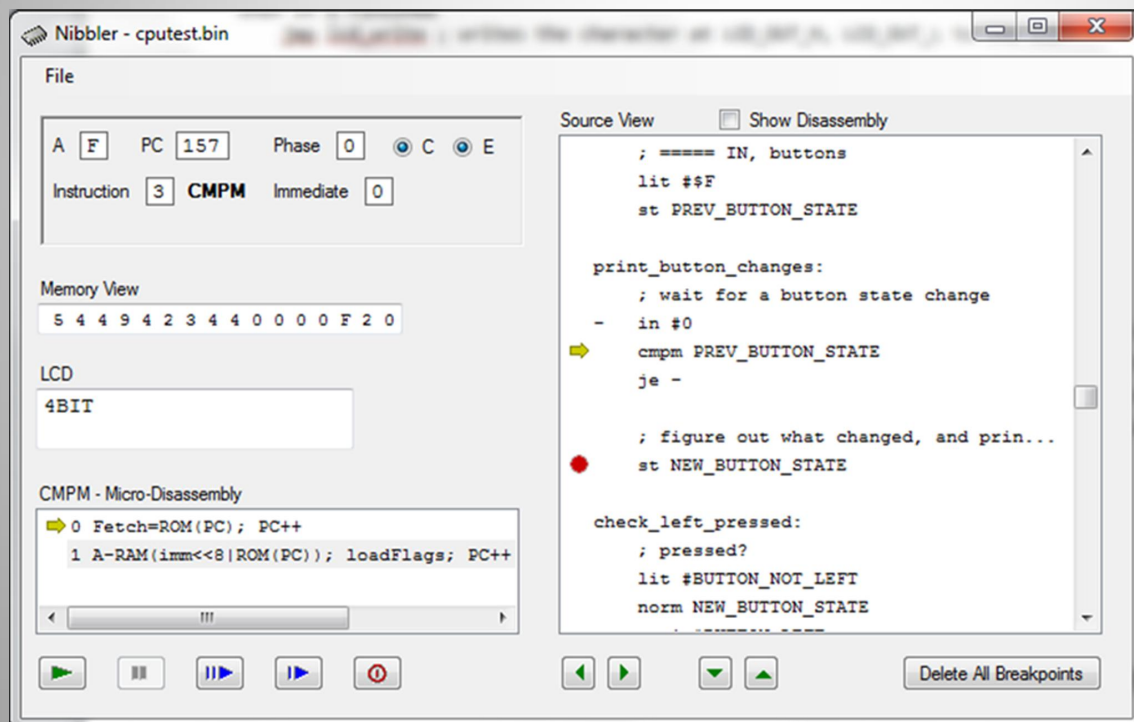
- 这些十六进制数只是微码表中的行, 16个控制信号表示为4个半字节的十六进制值。

(2) Assembler汇编器

- **Nibbler汇编器**
- **用来为CPU编写程序，使用符号命名指令名称和分支标记**
 - 汇编器作为命令行程序运行
 - 将一个.asm文件作为输入，并将代码汇编成.bin二进制输出文件
 - 还生成一个.Sym符号文件，包含代码中所有标签和常量的值，以及每个代码行的汇编地址
 - 符号文件稍后由模拟器使用，以便执行源级调试。
- **汇编器没有任何类似宏或条件编译的特殊功能**

(3) Simulator仿真器

- 机器仿真器是最复杂的工具
- 基于GUI图形的工具模拟CPU的数据和控制路径
 - 它支持源级符号调试、反汇编、微码调试、代码断点、数据断点、存储器检查以及LCD和输入按键的I/O仿真。



仿真模拟器使用程序二进制、程序源和符号文件（如果有）和微代码ROM二进制文件。

在上面的图像中，仿真器正在对按键观察者示例程序进行源级调试。

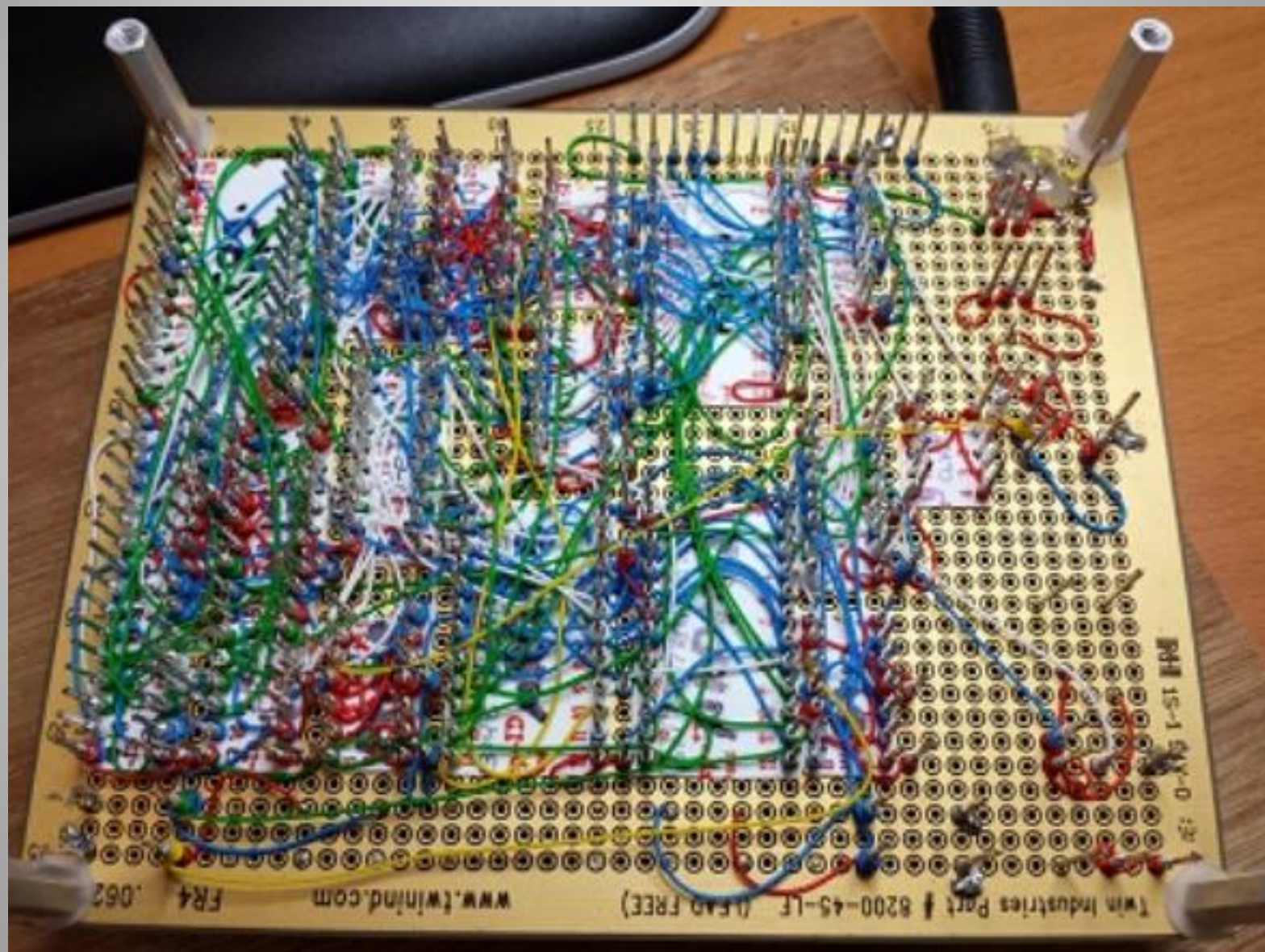
当前执行CMPM PREV_BUTTON_STATE指令执行已停止，如黄色箭头所示。在两个指令后有红色圆圈的行，表示有一个断点。如果模拟器启动并且模拟按钮被按下，程序将从循环中中断并在断点处停止。（图像中的JE指令后来被替换为JZ）。

PCB版Nibbler照片



- PCB版Nibbler电路图和设计文件请参考
- <https://github.com/pong74ls/Nibbler>
- Nibbler所有的设计文件和工具
- <http://www.bigmessowires.com/nibbler.zip>

手工洞洞版Nibbler反面



Nibbler元件清单

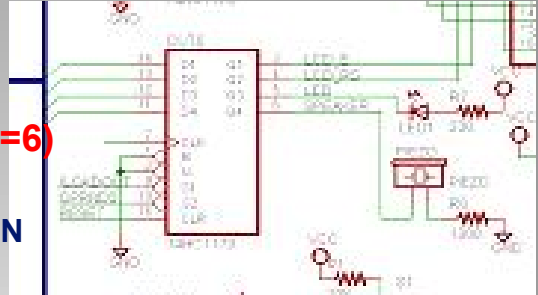
Category	Description	Reference	Quantity	Part Number
Capacitor	0.1uF 50V Ceramic	C1-C18	18	
Capacitor	470uf 16V Electrolytic	C19	1	
Capacitor	1uF 16V Tantalum	C20	1	
Diode	1N4148 Small Signal Diode	D1	1	
Resistor	1kΩ 5% 1/4W Carbon Film	R1, R8, R9	3	
Resistor	10kΩ 5% 1/4W Carbon Film	R3-R7, R10-R14	10	
Resistor	100kΩ 5% 1/4W Carbon Film	R2	1	
Resistor	68Ω 5% 1/4W Carbon Film	R15	1	
IC	Oscillator 2.46MHz	U1	1	SG-531P2.4576MC
IC	74HC74 Dual D-Type Flip-Flop	U2	1	
IC	74HC377 8-bit Register with Clock Enable	U3	1	
IC	128kB 70ns Flash ROM, 32-pin 0.6" DIP	U4, U5, U9	3	SST39SF010A-70-4C-PHE
IC	74HC163 4-bit Binary Counter with Synchronous Clear	U6-U8	3	
IC	74HC173 4-bit Register with 3-State Outputs	U10, U13, U17, U18	4	
IC	74HC181 4-bit Arithmetic Logic Unit	U11	1	
IC	74HC32 Quad 2-Input OR Gate	U12	1	
IC	74HC244 Dual Quad Buffer with 3-State Outputs	U14	1	
IC	32kB Static RAM, 28-pin 0.3" DIP	U15	1	CY7C199CN-15PXC
IC	74HC125 Quad Buffer with 3-State Outputs	U16	1	
LED	5mm LED	LED1, LED2	2	
Misc.	5.5x2.1 mm DC Power Jack	CON1	1	PJ-002A
Misc.	LCD Display Module	LCD1	1	NHD-0216K1Z-FL-YBW
Misc.	Piezoelectric Transducer	PZ1	1	PS1240P02BT
Misc.	10kΩ Trimmer Potentiometer	RV1	1	3306F-1-103
Misc.	Toggle Switch	SW1	1	200MSP1T1B1M2QEH
Misc.	12x12mm Tactile Pushbutton Switch	SW2-SW6	5	B3F-4050

Programming

- As our CPU is basically done, we need to program it to make something useful.
- Lets start with simple program.
 - **JMP** – 无条件转移
 - **JC** – 如果进位则跳转
 - **JNC** – 如果无进位则跳转
 - **JZ** – 如果零则跳转
 - **JNZ** – 如果非零则跳转
 - **LD**和**ST**指令
 - **LIT**指令--加载立即数到累加器
 - **OUT**指令
 - **IN**指令
 - **ADDI**指令-加立即数值到累加器
 - **ADDM**指令-加RAM值到累加器
 - **CMPI**指令--立即数与累加器比较(累加器减去数值)
 - **CMPM**--将ram与累加器比较
 - **NORI**指令--与立即数的NOR(或非)
 - **NORM**--与RAM值的NOR
 - 进位输出(carry-out)和零标志是由NOR的结果设置

type	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
addressed	i3	i2	i1	i0	a11	a10	a9	a8	a7	a6	a5	a4	a3	a2	a1	a0
immediate	i3	i2	i1	i0	o3	o2	o1	o0								

代码分析例---testram.asm内存测试



```
; OUT ports 输出口 1110=0
#define PORT_CONTROL $E
```

```
; memory locations 内存位置
#define TEST_LOCATION $038
```

init:

```
; turn LED on 亮灯
lit #$0 ;加载0000到A
out #PORT_CONTROL
```

testram:

```
lit #0 ;A=0
```

```
addi #15 ;A+=15 (1111)
st TEST_LOCATION ;store
cmpm TEST_LOCATION ;比较
jnz fail ;非零则跳
```

```
addi #15 ;A=30 (1110=14)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail
```

```
addi #15 ;A=29 (1101=13)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail
```

```
addi #15 ;A=28 (1100=12)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail
```

```
addi #15 ;A=27 (1011=11)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail
```

```
addi #15 ;A=26 (1010=10)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail
```

```
addi #15 ;A=25 (1001=9)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail
```

```
addi #15 ;A=24 (1000=8)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail
```

```
addi #15 ;A=23 (0111=7)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail
```

```
addi #15 ;A=22 (0110=6)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail
```

```
addi #15 ;A=21 (0101=5)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail
```

```
addi #15 ;A=20 (0100=4)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail
```

```
addi #15 ;A=19 (0011=3)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail
```

```
addi #15 ;A=18 (0010=2)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail
```

```
addi #15 ;A=17 (0001=1)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail
```

```
addi #15 ;A=16 (0000=0)
st TEST_LOCATION
cmpm
TEST_LOCATION
jnz fail
```

```
jmp testram ;死循环
```

fail:

```
; turn LED off
lit #4 ;A=0100
out #PORT_CONTROL
jmp fail
```

```
unreachable:
jmp fail
```

代码地址分析---testram.asm

```
L_testram.asm_2  = $0 ;
L_testram.asm_5  = $0 ;
L_testram.asm_7   = $0 ;
L_testram.asm_9   = $0 ;
L_testram.asm_10  = $1 ;
L_testram.asm_12  = $2 ;
L_testram.asm_13  = $2 ;
L_testram.asm_15  = $3 ;
L_testram.asm_16  = $4 ;
L_testram.asm_17  = $6 ;
L_testram.asm_18  = $8 ;
L_testram.asm_20  = $A ;
L_testram.asm_21  = $B ;
L_testram.asm_22  = $D ;
L_testram.asm_23  = $F ;
L_testram.asm_25  = $11 ;
L_testram.asm_26  = $12 ;
L_testram.asm_27  = $14 ;
L_testram.asm_28  = $16 ;
L_testram.asm_30  = $18 ;
L_testram.asm_31  = $19 ;
L_testram.asm_32  = $1B ;
L_testram.asm_33  = $1D ;
L_testram.asm_35  = $1F ;
L_testram.asm_36  = $20 ;
L_testram.asm_37  = $22 ;
L_testram.asm_38  = $24 ;
L_testram.asm_40  = $26 ;
L_testram.asm_41  = $27 ;
L_testram.asm_42  = $29 ;
L_testram.asm_43  = $2B ;
L_testram.asm_45  = $2D ;
L_testram.asm_46  = $2E ;
L_testram.asm_47  = $30 ;
L_testram.asm_48  = $32 ;
L_testram.asm_50  = $34 ;
L_testram.asm_51  = $35 ;
L_testram.asm_52  = $37 ;
L_testram.asm_53  = $39 ;
L_testram.asm_55  = $3B ;
L_testram.asm_56  = $3C ;
L_testram.asm_57  = $3E ;
L_testram.asm_58  = $40 ;
L_testram.asm_60  = $42 ;
L_testram.asm_61  = $43 ;
L_testram.asm_62  = $45 ;
L_testram.asm_63  = $47 ;
L_testram.asm_65  = $49 ;
L_testram.asm_66  = $4A ;
L_testram.asm_67  = $4C ;
L_testram.asm_68  = $4E ;
L_testram.asm_70  = $50 ;
L_testram.asm_71  = $51 ;
L_testram.asm_72  = $53 ;
```

```
L_testram.asm_73  = $55 ;
L_testram.asm_75  = $57 ;
L_testram.asm_76  = $58 ;
L_testram.asm_77  = $5A ;
L_testram.asm_78  = $5C ;
L_testram.asm_80  = $5E ;
L_testram.asm_81  = $5F ;
L_testram.asm_82  = $61 ;
L_testram.asm_83  = $63 ;
L_testram.asm_85  = $65 ;
L_testram.asm_86  = $66 ;
L_testram.asm_87  = $68 ;
L_testram.asm_88  = $6A ;
L_testram.asm_90  = $6C ;
L_testram.asm_91  = $6D ;
L_testram.asm_92  = $6F ;
L_testram.asm_93  = $71 ;
L_testram.asm_95  = $73 ;
L_testram.asm_97  = $75 ;
L_testram.asm_99  = $75 ;
L_testram.asm_100 = $76 ;
L_testram.asm_101 = $77 ;
L_testram.asm_103 = $79 ;
L_testram.asm_104 = $79 ;
PORT_CONTROL     = $E ;
TEST_LOCATION     = $38 ;
fail              = $75 ;
init              = $0 ;
testram           = $2 ;
unreachable       = $79 ;
```

代码汇编--testram.asm

```
; OUT ports 输出口 1110=0
#define PORT_CONTROL $E

; memory locations 内存位置
#define TEST_LOCATION $038

init: ($000)
; turn LED on 亮灯
lit #$0 ;0100 0000 ($40)
out #PORT_CONTROL
;1101 1110 ($DE)

testram: ($002)
lit #0 ;0100 0000 ($40)

addi #15 ;1010 1111 (AF)
st TEST_LOCATION ;0111 $038
; ($70 $38)
cmpm TEST_LOCATION ;0011
;($30 $38)
jnz fail ;1001 ($9X $ XX)

addi #15 ; 1010 1111
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail

addi #15 ; 1010 1111
st TEST_LOCATION ;0111
cmpm TEST_LOCATION ;0011
jnz fail

addi #15 ;A=28 (1100=12)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail

addi #15 ;A=27 (1011=11)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail

addi #15 ;A=26 (1010=10)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail

addi #15 ;A=25 (1001=9)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail

addi #15 ;A=24 (1000=8)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail

addi #15 ;A=23 (0111=7)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail

addi #15 ;A=22 (0110=6)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail

addi #15 ;A=21 (0101=5)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail

addi #15 ;A=20 (0100=4)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail

addi #15 ;A=19 (0011=3)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail

addi #15 ;A=18 (0010=2)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail

addi #15 ;A=17 (0001=1)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail

addi #15 ;A=16 (0000=0)
st TEST_LOCATION
cmpm TEST_LOCATION
jnz fail

jmp testram ;1100 $002
($C002)

fail: ($075)
; turn LED off
lit #4 ;0100 0100($44)
out #PORT_CONTROL
;1101 1110 ($DE)
jmp fail ;1100 $075
($C075)

unreachable:
jmp fail
```

机器码---testram.bin

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	
00000000h:	40	DE	40	AF	70	38	30	38	90	75	AF	70	38	30	38	90	; 8080808080808080
00000010h:	75	AF	70	38	30	38	90	75	AF	70	38	30	38	90	75	AF	; u8080808080808080
00000020h:	70	38	30	38	90	75	AF	70	38	30	38	90	75	AF	70	38	; p8080808080808080
00000030h:	30	38	90	75	AF	70	38	30	38	90	75	AF	70	38	30	38	; 0808080808080808
00000040h:	90	75	AF	70	38	30	38	90	75	AF	70	38	30	38	90	75	; 8080808080808080
00000050h:	AF	70	38	30	38	90	75	AF	70	38	30	38	90	75	AF	70	; 8080808080808080
00000060h:	38	30	38	90	75	AF	70	38	30	38	90	75	AF	70	38	30	; 8080808080808080
00000070h:	38	90	75	C0	02	44	DE	C0	75	C0	75	00	00	00	00	00	; 8080808080808080
00000080h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000090h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000a0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000b0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000c0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000d0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000e0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000f0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000100h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000110h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000120h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000130h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000140h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000150h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000160h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;

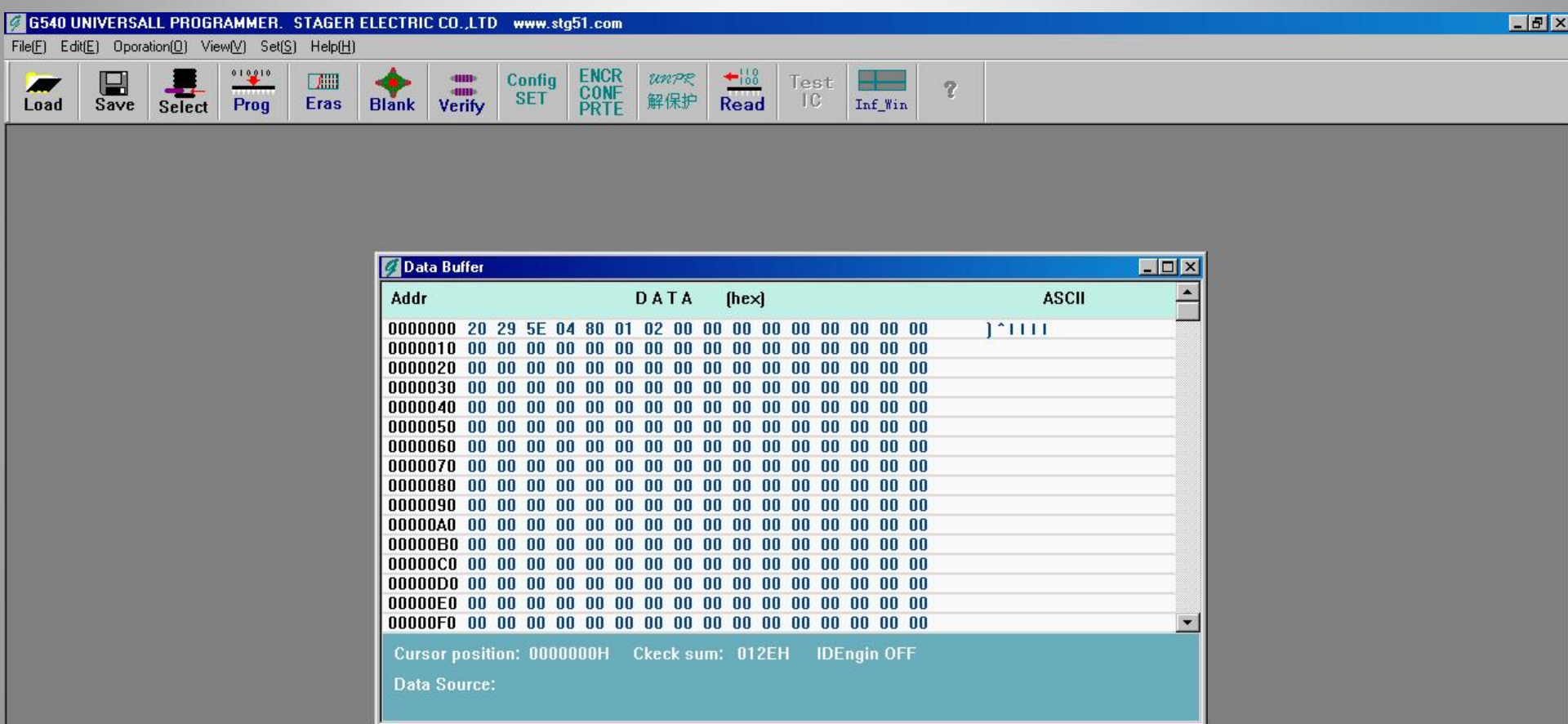
微码---microcode 1,0

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	
00000000h:	F8	38	F8	38	F8	F8	F8	F8	F8	F8	F8	F8	F8	38	F8	38	; 38? ??
00000010h:	F8	61	F8	61	F8	61	F8	61	F8	E1	F8	E1	F8	E1	F8	E1	; 呜呜呜呜
00000020h:	F8	46	F8	46	F8	46	F8	46	F8	46	F8	46	F8	46	F8	46	; 呜呜呜呜呜呜呜呜
00000030h:	F8	C6	F8	C6	F8	C6	F8	C6	F8	F8	F8	F8	F8	F8	F8	F8	;
00000040h:	F8	38	F8	F8	F8	38	F8	F8	F8	F8	38	F8	F8	F8	F8	38	; ? ? ? ?
00000050h:	F8	4A	F8	4A	F8	4A	F8	4A	F8	CA	F8	CA	F8	CA	F8	CA	; 呜呜呜呜
00000060h:	F8	30	F8	30	F8	30	F8	30	F8	78	F8	78	F8	78	F8	78	; ???? 呜呜呜呜
00000070h:	F8	44	F8	44	F8	44	F8	44	F8	C4	F8	C4	F8	C4	F8	C4	; 呜呜呜呜
00000080h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000090h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000a0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000b0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000c0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000d0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000e0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000f0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000100h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000110h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000120h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000130h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000140h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000150h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000160h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	
00000000h:	37	37	37	37	37	37	37	37	37	37	37	37	37	37	37	37	; 7777777777777777
00000010h:	37	BD	37	BD	37	BD	37	BD	37	9F	37	9F	37	9F	37	9F	; 7?????????
00000020h:	37	BD	37	BD	37	BD	37	BD	37	BB	37	BB	37	BB	37	BB	; 7?????????
00000030h:	37	9F	37	9F	37	9F	37	9F	37	07	37	07	37	07	37	07	; 7?????.7.7.7.
00000040h:	37	37	37	37	37	37	37	37	37	37	37	37	37	37	37	37	; 7777777777777777
00000050h:	37	7D	37	7D	37	7D	37	7D	37	5F	37	5F	37	5F	37	5F	; 7}7}7}7}7_7_7_7_
00000060h:	37	37	37	37	37	37	37	37	37	36	37	36	37	36	37	36	; 77777777776767676
00000070h:	37	7D	37	7D	37	7D	37	7D	37	5F	37	5F	37	5F	37	5F	; 7}7}7}7}7_7_7_7_
00000080h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000090h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000a0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000b0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000c0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000d0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000e0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000f0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000100h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000110h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000120h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000130h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000140h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000150h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000160h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;

Burning烧录代码

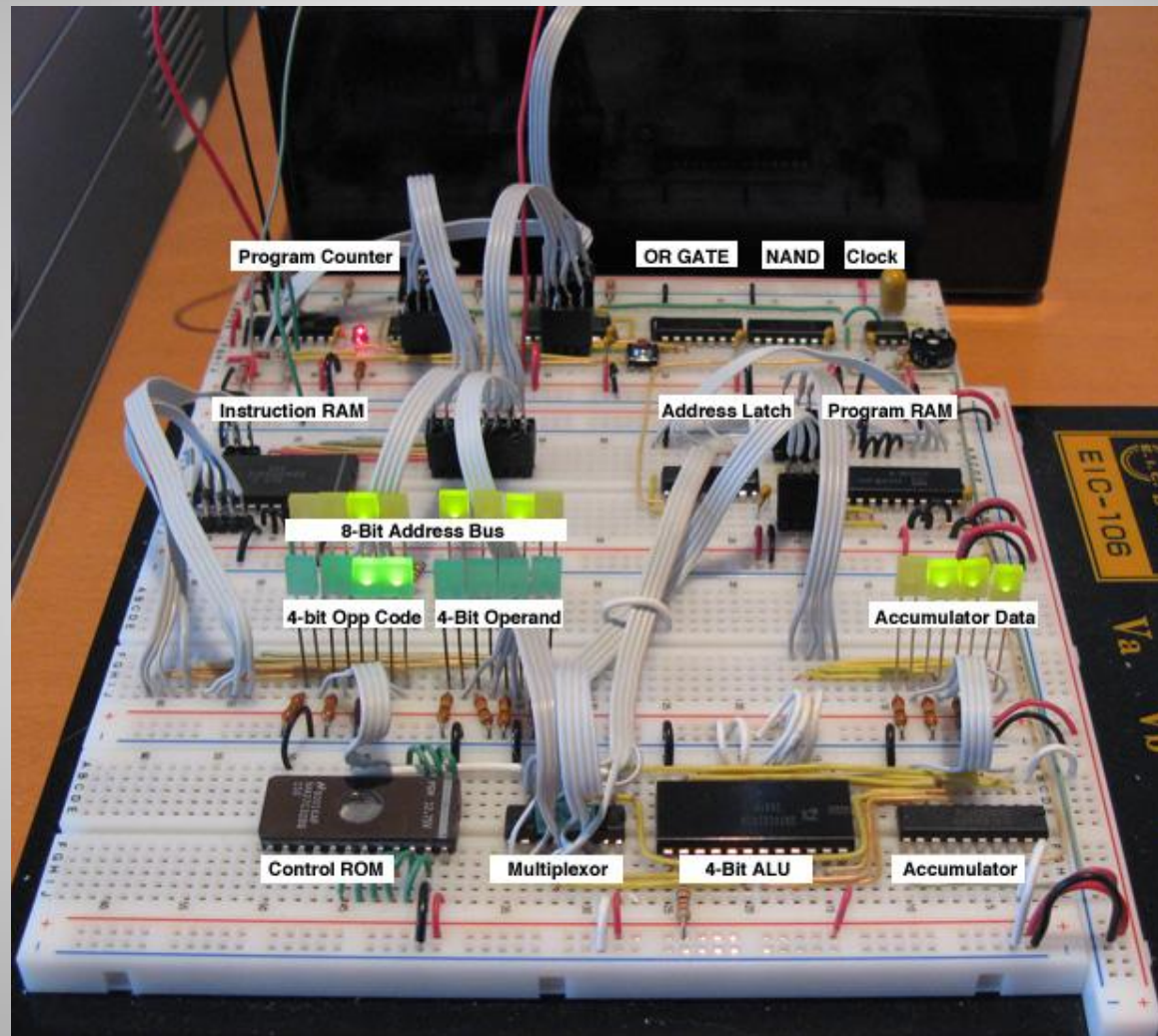
- That is ready to be burned into EEPROM.
 - use programmer



硬连线型SCHEMATIC



Another 4-bit TTL CPU



Nibbler Verilog

- 见2019版讲义
- Nibbler.v微处理器p106-/2019版
- Nibbler_ALU.v算术逻辑单元p109-/2019版
- 课后练习：
 - 下载Nibbler所有的设计文件和工具,
 - 找一个相关代码, 进行编译, 并在Nibbler仿真器中跑一遍。

课后作业

- 1) 查阅:

- 国际电路公司的CPU、MCU、DSP:
 - » 8051, PIC, AVR单片机, ARM, MIPS, DSP等微控制器芯片的型号、类型、速度.....
- 图书馆资源: 电子器件天地, 软件
- <ftp://10.13.71.58>

- 2) 作业:

- 学在浙大

- 3) 阅读:

- 复习《数字系统设计与Verilog HDL》
- <http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/>

课后作业

- 4) Project

- Project2（选做平时成绩加分）：用具体逻辑电路描述设计并FPGA实现一个4bit简单CPU实物。（期末考试停课前一周交）